== Description：

今天做的这个比赛问题是一个工业化生产预测的问题，任务目标是利用异烟酸生产过程中的各参数，
预测最终异烟酸的收率。数据集包括生产过程中 10 个步骤的参数，
样本 id、A1-A28、B1-B14 包括原料，辅料，时间，温度，压强及收率等。
可以看一下生产过程中的流程图： image:1.png[image]
这节课会学习数据检查和问题的修正，
如何根据原来的数据构建特征（特征工程），
时间段特征如何处理，时间特征应该怎么利用，
最后使用 xgboost 模型对异烟酸的收率进行预测。 这次做的是一个回归任务。

+*In[1]:*+
[source, ipython3]

"""导入包"""
import numpy as np
import pandas as pd

import pandas_profiling as ppf
import xgboost as xgb
from sklearn.model_selection import KFold
from sklearn.metrics import mean_squared_error as mse

import warnings
warnings.filterwarnings('ignore')

== 导入数据集并进行相关处理

我们使用的数据集是工业上直接进行记录的数据集，但是我们使用 Excel 观察一下数据集之后，就会发现一些格式上的问题，那么我们相应的就必须做出处理，关于数据集的检查这块，没有什么特别好的方式，毕竟不同的任务数据不同，只能是先打开数据大体上观察一遍，找到问题所在，当然也可以编写程序，遍历去观察等。我们下面先导入数据集，然后进行数据的检查和问题的修正工作

+*In[2]:*+
[source, ipython3]

"""导入数据集"""
df_trn = pd.read_csv('data/jinnan_round1_train_20181227.csv', encoding='GB2312') # 如何没有后面的编码信息会报错

```
df_tst_a     =     pd.read_csv('data/jinnan_round1_testA_20181227.csv',
encoding='GB2312')
df_tst_b     =     pd.read_csv('data/jinnan_round1_testB_20190121.csv',
encoding='GB2312')  # 测试集在这会用不到，也可以不导入
```

+*In[3]:*+

```
[source, ipython3]
```

```
"""观察数据"""
df_trn.head()
```

+*Out[3]:*+

```
[cols=",,,,,,,,,,,,,,,,,,,,,,",options="header",]
```

|=======

| |样本 id |A1 |A2 |A3 |A4 |A5 |A6 |A7 |A8 |A9 |... |B6 |B7 |B8 |B9 |B10 |B11 |B12 |B13 |B14 |收率

|0 |sample_1528 |300 |NaN |405.0 |700 |13:30:00 |38.0 |NaN |NaN |15:30:00 |... |65 |11:30:00 |45.0 |11:30-13:00 |14:00-15:30 |NaN |800.0 |0.15 |400 |0.879

|1 |sample_1698 |300 |NaN |405.0 |700 |14:00:00 |29.0 |NaN |NaN |16:00:00 |... |80 |6:00:00 |45.0 |6:00-7:30 |7:30-9:00 |9:00-10:00 |1200.0 |0.15 |400 |0.902

|2 |sample_639 |300 |NaN |405.0 |700 |14:00:00 |29.0 |NaN |NaN |16:00:00 |... |80 |1:00:00 |45.0 |1:00-2:30 |2:30-4:00 |4:00-5:00 |1200.0 |0.15 |400 |0.936

|3 |sample_483 |300 |NaN |405.0 |700 |1:30:00 |38.0 |NaN |NaN |3:00:00 |... |65 |18:00:00 |45.0 |19:00-20:30 |21:30-23:00 |NaN |800.0 |0.15 |400 |0.902

|4 |sample_617 |300 |NaN |405.0 |700 |22:00:00 |29.0 |NaN |NaN |0:00:00 |... |80 |9:00:00 |45.0 |9:00-10:30 |10:30-12:00 |12:00-13:00 |1200.0 |0.15 |420 |0.983

|=======

5 rows × 44 columns

+*In[4]:*+
[source, ipython3]
---
df_trn.info()
"""
1396 个训练样本， 44 列， 最后一列是 label， 有很多都存在缺失值， 会发现数据量是比较少的，
数据量少也可以做一些人为的数据增强之类的。
"""

+*Out[4]:*+

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1396 entries, 0 to 1395
Data columns (total 44 columns):
样本 id   1396 non-null object
A1   1396 non-null int64
A2   42 non-null float64
A3   1354 non-null float64
A4   1396 non-null int64
A5   1396 non-null object
A6   1396 non-null float64
A7   149 non-null object
A8   149 non-null float64
A9   1396 non-null object
A10   1396 non-null int64
A11   1396 non-null object
A12   1396 non-null int64
A13   1396 non-null float64
A14   1396 non-null object
A15   1396 non-null float64
A16   1396 non-null object
A17   1396 non-null float64
A18   1396 non-null float64
A19   1396 non-null int64
A20   1396 non-null object
A21   1393 non-null float64
A22   1396 non-null float64

```
A23     1393 non-null float64
A24     1395 non-null object
A25     1396 non-null object
A26     1394 non-null object
A27     1396 non-null int64
A28     1396 non-null object
B1      1386 non-null float64
B2      1394 non-null float64
B3      1394 non-null float64
B4      1396 non-null object
B5      1395 non-null object
B6      1396 non-null int64
B7      1396 non-null object
B8      1395 non-null float64
B9      1396 non-null object
B10     1152 non-null object
B11     547 non-null object
B12     1395 non-null float64
B13     1395 non-null float64
B14     1396 non-null int64
收率       1396 non-null float64
dtypes: float64(18), int64(8), object(18)
memory usage: 480.0+ KB
'\n1396 个训练样本， 44 列， 最后一列是 label， 有很多都存在缺失值， 会发现数据量是比较少的，\n 数据量少也可以做一些人为的数据增强之类的。\n'----
```

=== 数据的探索性分析

可以直接利用 ProfileReport 生成一份数据探索性报告， 在这里面会看到： * 总体的数据信息（首先是数据集信息：变量数(列)、观察数(行)、数据缺失率、内存；数据类型的分布情况），
* 警告信息 > * 要点：类型，唯一值，缺失值 > *
分位数统计量，如最小值，Q1，中位数，Q3，最大值，范围，四分位数范围 > * 描述性统计数据，如均值，模式，标准差，总和，中位数绝对偏差，变异系数，峰度，偏度

* 单变量描述（对每一个变量进行描述）
* 相关性分析（皮尔逊系数和斯皮尔曼系数）
* 采样查看等

+*In[5]:*+
[source, ipython3]

```
"""数据探索性分析"""
ppf.ProfileReport(df_trn)
```

+*Out[5]:*+

```
```

通过查看，我们初步获得的信息： 数据缺失问题，字段相关问题，
时间字段的问题，高基数警告（列中有很多唯一值），
44 列 1396 个样本,数据量也是比较少的，这种情况下也可以考虑人为做一些数据的增强。

=== 数据的检查与问题的修正

拿到数据之后，首先应该对数据进行怀疑，这些数据中有没有一些有问题的量，每一列常规情况下应该是一些什么样的值，正常情况下可以画一个分布图或者箱线图等这种图，这样可以检测出一些离群点，这样就可以指定筛选的范围，去过滤不同的离群点。所以拿到数据之后，先对数据做一个检查，看看有没有什么样的问题，有问题，应该及时的做出修改。尤其是这种工业上的问题，很可能会由于记录时不仔细会出现数据的错误问题，要进行一个检查。
那怎么检查呢？
数据量小的时候可以通过 Excel 简单的看一下，因为 Excel 表里面有个套用表格样式，套用上之后就会发现每一列的取值情况，这时候可以具体看到取得值是不是合法。
数据量比较大的时候就需要使用 python 写一个函数了，
遍历所有的列，然后每一列输出取值的情况，这样来检查是不是取值合法，不合法的进行处理。
所以下面我写一个 python 函数来检查每一列的值是不是合法。尤其注意那种时间段的列

+*In[6]:*+

```
[source, ipython3]
```

```
# 这里可以遍历列，然后输出一下每一列取得是什么值，从这里面看看有什么不合适的值
for col in df_trn.columns[1:]:
    print(col, ": ", df_trn[col].value_counts().index)
```

+*Out[6]:*+

```
A1 :  Int64Index([300, 200, 250], dtype='int64')
A2 :  Float64Index([125.0], dtype='float64')
A3 :  Float64Index([405.0, 270.0, 340.0], dtype='float64')
A4 :  Int64Index([700, 980, 470, 590], dtype='int64')
A5 :  Index(['6:00:00', '14:00:00', '22:00:00', '21:00:00', '8:00:00',
'1:00:00',
       '18:00:00',  '2:00:00',  '17:00:00',  '15:00:00',  '12:30:00',
'11:00:00',
       '12:00:00',  '7:00:00',  '4:00:00',  '21:30:00',  '23:00:00',
'5:00:00',
       '0:00:00',  '10:00:00',  '9:00:00',  '19:00:00',  '6:30:00',
'13:00:00',
       '3:30:00',  '16:00:00',  '15:30:00',  '20:00:00',  '3:00:00',
'10:30:00',
       '17:30:00',  '14:30:00',  '7:30:00',  '13:30:00',  '2:30:00',
'16:30:00',
       '8:30:00',  '20:30:00',  '22:30:00',  '4:30:00',  '1:30:00',
'5:30:00',
       '19:30:00', '9:30:00', '18:30:00', '11:30:00', '15:40:00',
       '1900/1/21 0:00', '13:50:00', '23:30:00', '11:20:00', '2:45:00',
       '1900/1/29 0:00', '13:10:00', '10:45:00', '10:40:00', '14:50:00',
       '0:30:00',  '16:40:00',  '8:15:00',  '6:50:00',  '15:10:00',
'11:45:00',
       '8:45:00', '3:45:00', '7:45:00', '9:40:00'],
      dtype='object')
A6 :  Float64Index([29.0, 30.0, 21.0, 24.0, 27.0, 38.0, 28.0, 22.0,
23.0, 20.0, 39.0,
              18.0, 26.0, 37.0, 35.0, 34.0, 25.0, 40.0, 19.0, 36.0, 41.0,
80.0,
              33.0, 31.0, 32.0, 17.0, 50.0, 65.0, 70.0, 22.7, 44.0, 60.0,
42.0,
              43.0, 47.0, 48.0, 51.0, 72.0, 97.0],
             dtype='float64')
A7 :  Index(['12:40:00', '15:40:00', '2:10:00', '7:00:00', '21:40:00',
'2:30:00',
       '7:10:00',  '7:40:00',  '17:30:00',  '19:40:00',  '17:10:00',
'12:15:00',
       '12:30:00',  '2:45:00',  '15:20:00',  '21:45:00',  '6:15:00',
'15:10:00',
       '7:45:00',  '9:30:00',  '2:40:00',  '23:30:00',  '6:20:00',
'4:30:00',
```

'21:20:00', '22:00:00', '13:40:00', '15:45:00', '3:00:00', '22:30:00',
'13:00:00', '21:15:00', '4:20:00', '12:45:00', '16:00:00', '20:40:00',
'3:30:00', '5:30:00', '21:10:00', '21:30:00', '1:50:00', '8:40:00',
'6:40:00', '1:10:00', '14:40:00', '17:15:00', '2:20:00', '13:10:00',
'21:00:00', '7:30:00', '8:10:00', '22:20:00', '11:15:00', '6:30:00',
'18:10:00', '15:15:00', '16:40:00', '1:30:00', '13:45:00', '17:45:00',
'18:00:00', '20:15:00', '11:45:00', '16:20:00', '5:15:00', '12:00:00',
'22:40:00', '3:50:00', '9:00:00', '19:30:00', '14:15:00', '6:25:00',
'1:00:00', '3:40:00', '15:00:00'],
dtype='object')
A8 : Float64Index([80.0, 73.0, 74.0, 82.0, 75.0, 81.0, 70.0, 72.0], dtype='float64')
A9 : Index(['8:00:00', '16:00:00', '0:00:00', '23:00:00', '3:00:00', '19:00:00',
'23:30:00', '20:00:00', '10:00:00', '11:00:00', '15:00:00', '5:00:00',
'15:30:00', '14:00:00', '2:30:00', '7:00:00', '12:30:00', '7:30:00',
'4:00:00', '17:00:00', '22:30:00', '13:30:00', '9:00:00', '17:30:00',
'14:30:00', '6:00:00', '18:30:00', '16:30:00', '18:00:00', '9:30:00',
'11:30:00', '22:00:00', '6:30:00', '1:30:00', '8:30:00', '13:00:00',
'3:30:00', '21:00:00', '4:30:00', '12:00:00', '0:30:00', '10:30:00',
'5:30:00', '1:00:00', '19:30:00', '20:30:00', '2:00:00', '21:30:00',
'8:20:00', '8:15:00', '18:40:00', '14:40:00', '9:40:00', '17:15:00',
'23:20:00', '5:45:00', '7:40:00', '14:20:00', '9:20:00', '8:50:00',
'14:10:00', '3:10:00', '22:45:00', '23:15:00', '23:40:00', '19:50:00',
'13:40:00', '13:50:00', '8:05:00', '700', '4:15:00', '16:15:00',
'22:20:00', '10:35:00', '21:20:00', '2:40:00', '10:45:00',

'7:10:00',
'16:50:00', '16:20:00', '14:50:00', '9:10:00', '6:50:00', '11:50:00',
'17:50:00', '4:40:00', '8:45:00', '13:20:00', '15:40:00', '17:20:00',
'14:15:00', '17:10:00', '1900/1/9 7:00', '18:05:00', '15:20:00'],
dtype='object')
A10 : Int64Index([100, 102, 101, 103], dtype='int64')
A11 : Index(['9:00:00', '17:00:00', '1:00:00', '0:00:00', '4:00:00', '20:00:00',
'21:00:00', '11:00:00', '0:30:00', '16:00:00', '12:00:00', '8:00:00',
'6:00:00', '3:30:00', '16:30:00', '15:00:00', '13:30:00', '8:30:00',
'23:30:00', '5:00:00', '14:30:00', '18:00:00', '17:30:00', '7:00:00',
'18:30:00', '19:00:00', '10:00:00', '15:30:00', '7:30:00', '19:30:00',
'10:30:00', '12:30:00', '2:30:00', '9:30:00', '23:00:00', '22:00:00',
'13:00:00', '4:30:00', '5:30:00', '14:00:00', '1:30:00', '6:30:00',
'11:30:00', '2:00:00', '20:30:00', '3:00:00', '9:20:00', '21:30:00',
'22:30:00', '9:15:00', '19:40:00', '6:45:00', '8:40:00', '9:50:00',
'10:40:00', '15:40:00', '10:20:00', '18:15:00', '15:20:00', '0:20:00',
'4:10:00', ':30:00', '7:50:00', '12:50:00', '14:40:00', '17:15:00',
'18:50:00', '8:10:00', '19:15:00', '5:40:00', '22:20:00', '0:15:00',
'18:20:00', '10:10:00', '1900/1/1 2:30', '20:50:00', '17:50:00',
'15:10:00', '15:15:00', '5:15:00', '0:40:00', '3:40:00', '15:50:00',
'16:40:00', '16:20:00', '11:45:00', '23:20:00', '23:45:00', '14:50:00',
'9:05:00', '11:35:00', '18:10:00', '12:15:00', '14:20:00'],
dtype='object')
A12 : Int64Index([103, 102, 104, 101, 105, 100, 106, 107, 98], dtype='int64')
A13 : Float64Index([0.2, 0.15, 0.12], dtype='float64')
A14 : Index(['10:00:00', '18:00:00', '2:00:00', '1:00:00', '5:00:00', '21:00:00',

'22:00:00', '12:00:00', '1:30:00', '13:00:00', '17:00:00',
'17:30:00',
'7:00:00', '9:00:00', '16:00:00', '14:30:00', '4:30:00',
'9:30:00',
'0:30:00', '6:00:00', '15:30:00', '19:30:00', '19:00:00',
'16:30:00',
'18:30:00', '11:00:00', '8:00:00', '8:30:00', '20:00:00',
'11:30:00',
'20:30:00', '0:00:00', '3:30:00', '13:30:00', '14:00:00',
'5:30:00',
'23:00:00', '10:30:00', '2:30:00', '6:30:00', '15:00:00',
'12:30:00',
'7:30:00', '3:00:00', '21:30:00', '10:20:00', '22:30:00',
'4:00:00',
'23:30:00', '10:15:00', '20:40:00', '16:40:00', '19:15:00',
'10:50:00',
'11:40:00', '11:20:00', '1:20:00', '16:20:00', '9:40:00',
'7:45:00',
'9:10:00', '6:40:00', '17:20:00', '6:15:00', '12:35:00',
'11:10:00',
'23:20:00', '19:10:00', '10:05:00', '13:50:00', '16:15:00',
'17:40:00',
'21:50:00', '12:45:00', '15:50:00', '16:10:00', '18:50:00',
'19:50:00',
'5:10:00', '13:15:00', '20:15:00', '16:50:00', '0:45:00',
'19:20:00',
'15:40:00', '1:40:00', '8:50:00', '4:40:00', '1:15:00',
'18:15:00',
'0:20:00', '15:20:00'],
dtype='object')
A15 : Float64Index([104.0, 103.0, 105.0, 102.0, 106.0, 101.0, 107.0,
103.8, 100.0,
109.0],
dtype='float64')
A16 : Index(['11:00:00', '19:00:00', '3:00:00', '2:00:00', '6:00:00',
'23:00:00',
'22:00:00', '13:00:00', '2:30:00', '18:00:00', '14:00:00',
'8:00:00',
'17:00:00', '5:30:00', '18:30:00', '10:00:00', '15:30:00',
'10:30:00',
'16:30:00', '7:00:00', '1:30:00', '20:30:00', '19:30:00',
'17:30:00',
'20:00:00', '9:00:00', '12:00:00', '21:00:00', '9:30:00',
'21:30:00',

```
       '1:00:00',  '12:30:00',  '11:30:00',  '4:30:00',  '14:30:00',
'3:30:00',
       '0:00:00',  '15:00:00',  '6:30:00',  '7:30:00',  '16:00:00',
'8:30:00',
       '22:30:00',  '13:30:00',  '4:00:00',  '23:30:00',  '5:00:00',
'11:20:00',
       '0:30:00',  '11:15:00',  '21:40:00',  '10:40:00',  '8:45:00',
'11:50:00',
       '17:20:00',  '12:40:00',  '12:20:00',  '20:15:00',  '17:40:00',
'7:15:00',
       '17:15:00',  '0:20:00',  '18:20:00',  '7:40:00',  '5:40:00',
'11:05:00',
       '2:40:00',  '10:10:00',  '21:15:00',  '2:15:00',  '19:50:00',
'9:50:00',
       '17:10:00', '18:40:00', '1900/1/12 0:00', '1:10:00', '16:50:00',
       '16:20:00',  '20:10:00',  '19:15:00',  '13:45:00',  '16:40:00',
'17:50:00',
       '14:50:00',  '1:45:00',  '1:20:00',  '22:50:00',  '13:35:00',
'14:15:00',
       '2:20:00', '20:50:00', '6:10:00', '12:10:00', '20:20:00'],
      dtype='object')
A17 :  Float64Index([105.0, 104.0, 106.0, 102.0, 107.0, 103.0, 108.0,
101.0, 103.5,
              101.4, 103.2,  89.0, 100.0],
             dtype='float64')
A18 :  Float64Index([0.2, 0.1], dtype='float64')
A19 :  Int64Index([200, 300, 100, 150, 350, 320], dtype='int64')
A20 :  Index(['11:00-12:00', '19:00-20:00', '3:00-4:00', '2:00-3:00',
'11:30-12:00',
       '13:00-14:00', '16:00-16:30', '6:00-6:30', '6:00-7:00', '23:00-
0:00',
       ...
       '15:10-15:30', '23:20-23:50', '10:00-11:0', '12:45-13:15',
       '16:40-17:10', '19:15-19:30', '14:10-14:30', '14:20-14:50',
       '18:20-18:50', '5:30-6:30'],
      dtype='object', length=159)
A21 :  Float64Index([50.0, 40.0, 30.0, 35.0, 20.0, 60.0, 45.0, 55.0,
80.0, 70.0, 25.0,
              90.0],
             dtype='float64')
A22 :  Float64Index([9.0, 10.0, 8.0, 3.5], dtype='float64')
A23 :  Float64Index([5.0, 10.0, 4.0], dtype='float64')
A24 :  Index(['12:00:00', '20:00:00', '4:00:00', '3:00:00', '7:00:00',
'12:30:00',
```

```
       '0:00:00',  '14:00:00',  '15:00:00',  '19:00:00',  '17:00:00',
'1:30:00',
       '7:30:00',  '1:00:00',  '15:30:00',  '21:00:00',  '5:00:00',
'23:00:00',
       '16:30:00',  '16:00:00',  '11:30:00',  '11:00:00',  '18:00:00',
'6:00:00',
       '2:00:00',  '20:30:00',  '22:00:00',  '2:30:00',  '18:30:00',
'6:30:00',
       '23:30:00',  '14:30:00',  '5:30:00',  '4:30:00',  '19:30:00',
'13:00:00',
       '3:30:00',  '21:30:00',  '13:30:00',  '8:00:00',  '10:30:00',
'10:00:00',
       '0:30:00',  '9:00:00',  '17:30:00',  '22:30:00',  '9:30:00',
'12:20:00',
       '11:10:00',  '19:40:00',  '20:10:00',  '12:50:00',  '14:40:00',
'21:10:00',
       '23:45:00',  '21:20:00',  '11:15:00',  '13:10:00',  '17:25:00',
'17:20:00',
       '7:40:00',  '9:45:00',  '8:20:00',  '11:05:00',  '18:20:00',
'10:20:00',
       '14:15:00',  '7:20:00',  '6:20:00',  '16:40:00',  '1:20:00',
'19:20:00',
       '15:10:00',  '16:20:00',  '5:10:00',  '3:45:00',  '1:10:00',
'2:10:00',
       '0:20:00',  '11:20:00',  '20:15:00',  '14:50:00',  '12:40:00',
'0:40:00',
       '8:30:00',  '11:40:00',  '22:40:00',  '3:50:00',  '2:20:00',
'13:20:00',
       '15:20:00'],
      dtype='object')
A25 :  Index(['80', '70', '78', '79', '75', '71', '90', '72', '76',
'83', '85', '77',
       '73', '1900/3/10 0:00', '82'],
      dtype='object')
A26 :  Index(['13:00:00', '21:00:00', '5:00:00', '4:00:00', '8:00:00',
'15:00:00',
       '7:30:00',  '2:00:00',  '16:00:00',  '1:00:00',  '12:30:00',
'17:00:00',
       '19:30:00',  '23:30:00',  '5:30:00',  '17:30:00',  '11:30:00',
'15:30:00',
       '4:30:00',  '16:30:00',  '12:00:00',  '3:00:00',  '0:30:00',
'1:30:00',
       '7:00:00',  '3:30:00',  '20:00:00',  '18:30:00',  '19:00:00',
'22:30:00',
```

```
       '20:30:00',  '6:30:00',  '22:00:00',  '21:30:00',  '0:00:00',
'6:00:00',
       '14:00:00',  '18:00:00',  '13:30:00',  '2:30:00',  '14:30:00',
'11:00:00',
       '10:00:00',  '8:30:00',  '23:00:00',  '10:30:00',  '9:30:00',
'9:00:00',
       '11:40:00',  '17:55:00',  '17:10:00',  '18:50:00',  '4:50:00',
'12:50:00',
       '13:40:00', '1900/3/13 0:00', '21:40:00', '12:10:00', '11:35:00',
       '5:40:00',  '15:20:00',  '15:50:00',  '11:50:00',  '1:40:00',
'14:55:00',
       '21:50:00',  '15:01:00',  '10:15:00',  '13:50:00',  '21:45:00',
'4:45:00',
       '15:40:00',  '13:20:00',  '0:15:00',  '20:40:00',  '8:50:00',
'1:50:00',
       '6:50:00',  '16:50:00',  '11:45:00',  '10:50:00',  '19:50:00',
'0:50:00',
       '20:10:00', '2:50:00', '7:50:00', '17:50:00', '1:10:00'],
      dtype='object')
A27 :  Int64Index([73, 78, 75, 72, 76, 70, 77, 74, 79, 80, 45, 71, 55],
dtype='int64')
A28 :  Index(['13:00-14:00', '21:00-22:00', '5:00-6:00', '4:00-5:00',
'15:00-16:00',
       '8:00-9:00', '1:00-2:00', '19:30-20:00', '23:30-0:30', '3:00-
4:00',
       ...
       '21:30-22:00', '9:10-9:40', '8:30-9:30', '14:15-15:15', '14::30-
15:30',
       '16:00-17:30', '1:30-2:00', '22:00-23:30', '4:00-5:30', '18:50-
20:00'],
      dtype='object', length=157)
B1 :  Float64Index([ 320.0,  300.0,  350.0,  340.0,  370.0,  290.0,
310.0,  330.0,
               380.0,  360.0,  280.0, 1200.0,  190.0,  260.0,  240.0,
200.0,
               230.0,  270.0,  220.0,    3.5,  318.0],
             dtype='float64')
B2 :  Float64Index([3.5, 0.15, 3.6], dtype='float64')
B3 :  Float64Index([3.5, 3.6], dtype='float64')
B4 :  Index(['14:00-15:00', '22:00-23:00', '6:00-7:00', '5:00-6:00',
'9:00-10:00',
       '16:00-17:00', '2:00-3:00', '4:00-5:00', '18:00-19:00', '4:30-
5:30',
       ...
```

```
       '0:55-1:55', '8:40-9:40', '17:00-19:00', '9:10-10:10', '1:30-
3;00',
       '15:40-17:00', '8:30-11:00', '20:00-21:0', '11;30-13；00',
       '18:00-17:00'],
      dtype='object', length=178)
B5 :  Index(['15:00:00', '23:00:00', '7:00:00', '6:00:00', '13:00:00',
'17:00:00',
       '3:00:00',  '10:00:00',  '22:00:00',  '18:00:00',  '16:30:00',
'14:00:00',
       '19:00:00',  '5:00:00',  '12:30:00',  '5:30:00',  '7:30:00',
'1:30:00',
       '17:30:00',  '2:00:00',  '2:30:00',  '15:30:00',  '8:00:00',
'13:30:00',
       '21:30:00',  '0:00:00',  '21:00:00',  '6:30:00',  '18:30:00',
'20:00:00',
       '20:30:00',  '0:30:00',  '9:30:00',  '22:30:00',  '9:00:00',
'16:00:00',
       '4:30:00',  '4:00:00',  '1:00:00',  '3:30:00',  '12:00:00',
'11:00:00',
       '19:30:00',  '14:30:00',  '23:30:00',  '8:30:00',  '11:30:00',
'10:30:00',
       '23:40:00',  '22:20:00',  '15:20:00',  '1:55:00',  '12:15:00',
'7:50:00',
       '9:40:00',  '20:05:00',  '17:05:00',  '10:10:00',  '18:20:00',
'2:50:00'],
      dtype='object')
B6 :  Int64Index([80, 65, 60, 79, 78, 70, 50, 75, 68, 74, 45, 62, 76,
71, 52, 69, 66,
            64, 72, 67, 73, 58, 56, 63, 40, 55, 54, 48, 77, 57, 59, 53,
61, 51,
            49, 42],
           dtype='int64')
B7 :  Index(['17:00:00', '1:00:00', '9:00:00', '8:00:00', '19:00:00',
'12:00:00',
       '4:00:00',  '21:00:00',  '5:00:00',  '16:00:00',  '3:00:00',
'23:00:00',
       '0:00:00',  '19:30:00',  '6:00:00',  '10:00:00',  '10:30:00',
'20:00:00',
       '3:30:00',  '22:00:00',  '11:00:00',  '15:00:00',  '21:30:00',
'23:30:00',
       '15:30:00',  '20:30:00',  '14:00:00',  '13:00:00',  '16:30:00',
'7:30:00',
       '11:30:00',  '22:30:00',  '7:00:00',  '2:00:00',  '2:30:00',
'14:30:00',
```

```
       '17:30:00',  '18:00:00',  '8:30:00',  '5:30:00',  '0:30:00',
'6:30:00',
       '4:30:00',  '9:30:00',  '1:30:00',  '12:30:00',  '18:30:00',
'13:30:00',
       '17:40:00',  '7:20:00',  '22:45:00',  '17:20:00',  '17:10:00',
'17:05:00',
       '8:15:00', '1:20:00', '3:50:00', '14:20:00'],
      dtype='object')
B8 :  Float64Index([45.0, 40.0, 50.0, 28.0, 30.0, 46.0, 27.0, 38.0,
37.0, 36.0, 25.0,
              39.0, 26.0, 47.0, 35.0, 42.0, 22.0, 20.0, 32.0, 48.0, 24.0,
73.0,
              49.0, 29.0, 65.0],
             dtype='float64')
B9 :  Index(['17:00-18:30', '9:00-10:30', '1:00-2:30', '1:00-5:00',
'17:00-21:00',
       '8:00-9:30', '8:00-12:00', '19:00-20:30', '12:00-13:30', '5:00-
6:30',
       ...
       '20:10-21:20',  '12:10-13:40',  '17:10-18:40',  '19:00-20:45',
'0:00-3:30',
       '2:30-4:30', '23:40-1:00', '15:40-17:00', '1:00-2:00', '9:00-
12:30'],
      dtype='object', length=178)
B10 :  Index(['10:30-12:00', '18:30-20:00', '2:30-4:00', '10:00-11:30',
'13:30-15:00',
       '6:30-8:00', '21:30-23:00', '20:30-22:00', '23:00-0:30', '19:00-
20:30',
       ...
       '5:10-6；30', '2:00-3:00', '9:40-10:50', '4:00-6:00', '13:10-
14:30',
       '5:40-6:40', '14:30-16;00', '21:20-22:40', '21:40-22:50', '9:00-
10:20'],
      dtype='object', length=180)
B11 :  Index(['20:00-21:00', '12:00-13:00', '4:00-5:00', '22:00-23:00',
'15:00-16:00',
       '8:00-9:00', '9:00-10:00', '20:00-21:30', '11:00-12:00', '21:00-
22:00',
       '4:00-5:30', '12:00-13:30', '17:00-18:00', '23:00-0:00', '13:00-
14:00',
       '1:00-2:00', '10:00-11:00', '9:00-10:30', '22:00-12:00', '18:00-
19:00',
       '0:00-1:00', '14:00-15:00', '19:30-20:30', '3:00-4:00', '21:20-
22:20',
```

'8:30-10:30', '5:00-4:00', '2:30-3:30', '6:00-7:00', '10:30-11:30',
'23:00-0:30', '8:00-10:00', '8:45-10:00', '10:00-11:30', '0:30-1:00',
'20:10-21:30', '10:30-12:00'],
dtype='object')
B12 : Float64Index([1200.0, 800.0, 900.0, 400.0], dtype='float64')
B13 : Float64Index([0.15, 0.03, 0.06], dtype='float64')
B14 : Int64Index([400, 420, 440, 460, 385, 390, 380, 340, 280, 260, 40, 405, 395,
392, 387, 418, 370, 360, 350, 290, 256],
dtype='int64')
收率 : Float64Index([0.902, 0.93, 0.8909999999999999,
0.9470000000000001, 0.925, 0.935,
0.9359999999999999, 0.879, 0.9129999999999999,
0.983, 0.971, 0.943,
0.9229999999999999, 0.973, 0.9590000000000001,
0.9670000000000001, 0.89, 0.955,
0.888, 0.985, 0.9179999999999999,
0.98, 0.995, 0.868,
0.998, 0.92, 0.9,
0.8740000000000001, 0.8907, 0.9420000000000001,
0.9790000000000001, 0.8959999999999999, 0.884,
0.961, 0.87, 0.8340000000000001,
0.945, 0.9798, 0.9279999999999999,
0.919, 0.899, 0.9797,
0.882, 0.924, 0.956,
0.885, 0.941, 0.912,

0.846, 0.922,
0.677,
0.929, 0.8029999999999999,
0.9490000000000001,
1.0008, 0.9740000000000001,
0.624,
0.977, 0.95,
0.9620000000000001,
0.878, 0.897,
0.993,
0.866, 0.965,
0.958,
0.892, 0.909,
0.857,
0.871, 0.9259999999999999,
0.91,
0.9570000000000001],
dtype='float64')

+*In[7]:*+
[source, ipython3]

这一块没有什么固定的模板，需要自己去观察修正， 没有什么技术含量，但是必须去做的事情。

```
def train_abnormal_revise(data):
    df_trn = data.copy()                # 记得要复制一份处理，不要改变原来的数据
    df_trn.loc[(df_trn['A1'] == 200) & (df_trn['A3'] == 405), 'A1'] = 300   # 这样的就 1 个，可能是写错了，因为 300 对应 405, 毕竟这都是配料
    # A5 这一列，会发现有三个不合法的值，要替换掉
    df_trn['A5'] = df_trn['A5'].replace('1900/1/21 0:00', '21:00:00')
    df_trn['A5'] = df_trn['A5'].replace('1900/1/29 0:00', '14:00:00')
    df_trn['A9'] = df_trn['A9'].replace('1900/1/9 7:00', '23:00:00')
    # A9 有两个不合法的值
    df_trn['A9'] = df_trn['A9'].replace('1900/1/9 7:00', '23:00:00')
    df_trn['A9'] = df_trn['A9'].replace('700', '7:00:00')
    # A11 有一个不合法的值
    df_trn['A11'] = df_trn['A11'].replace('1900/1/1 2:30', '2:30:00')
    df_trn['A11'] = df_trn['A11'].replace(':30:00', '00:30:00')
    df_trn['A16'] = df_trn['A16'].replace('1900/1/12 0:00', '12:00:00')
    df_trn['A20'] = df_trn['A20'].replace('6:00-6:30 分', '6:00-6:30')
```

```
    df_trn['A20'] = df_trn['A20'].replace('18:30-15:00', '18:30-19:00')
    # A22 有个不合法的值，记错了应该是
    df_trn['A22'] = df_trn['A22'].replace(3.5, np.nan)
    df_trn['A25']    =    df_trn['A25'].replace('1900/3/10    0:00',
70).astype(int)
    df_trn['A26'] = df_trn['A26'].replace('1900/3/13 0:00', '13:00:00')
    df_trn['B1'] = df_trn['B1'].replace(3.5, np.nan)
    df_trn['B4'] = df_trn['B4'].replace('15:00-1600', '15:00-16:00')
    df_trn['B4'] = df_trn['B4'].replace('18:00-17:00', '16:00-17:00')
    df_trn['B4'] = df_trn['B4'].replace('19:-20:05', '19:05-20:05')
    df_trn['B9'] = df_trn['B9'].replace('23:00-7:30', '23:00-00:30')
    df_trn['B14'] = df_trn['B14'].replace(40, 400)

    return df_trn

def test_a_abnormal_revise(data):
    df_tst = data.copy()
    df_tst['A5'] = df_tst['A5'].replace('1900/1/22 0:00', '22:00:00')
    df_tst['A7'] = df_tst['A7'].replace('0:50:00', '21:50:00')
    df_tst['B14'] = df_tst['B14'].replace(785, 385)
    return df_tst

def train_abnormal_adjust(data):
    df_trn = data.copy()
    df_trn.loc[df_trn['样本 id'] == 'sample_1894', 'A5'] = '14:00:00'
    df_trn.loc[df_trn['样本 id'] == 'sample_1234', 'A9'] = '0:00:00'
    df_trn.loc[df_trn['样本 id'] == 'sample_1020', 'A9'] = '18:30:00'

    df_trn.loc[df_trn['样本 id'] == 'sample_1380', 'A11'] = '15:30:00'
    df_trn.loc[df_trn['样本 id'] == 'sample_844', 'A11'] = '10:00:00'
    df_trn.loc[df_trn['样本 id'] == 'sample_1348', 'A11'] = '17:00:00'
    df_trn.loc[df_trn['样本 id'] == 'sample_25', 'A11'] = '00:30:00'
    df_trn.loc[df_trn['样本 id'] == 'sample_1105', 'A11'] = '4:00:00'
    df_trn.loc[df_trn['样本 id'] == 'sample_313', 'A11'] = '15:30:00'

    df_trn.loc[df_trn['样本 id'] == 'sample_291', 'A14'] = '19:30:00'

    df_trn.loc[df_trn['样本 id'] == 'sample_1398', 'A16'] = '11:00:00'
    df_trn.loc[df_trn['样本 id'] == 'sample_1177', 'A20'] = '19:00-
20:00'

    df_trn.loc[df_trn['样本 id'] == 'sample_71', 'A20'] = '16:20-16:50'
    df_trn.loc[df_trn['样本 id'] == 'sample_14', 'A20'] = '18:00-18:30'
    df_trn.loc[df_trn['样本 id'] == 'sample_69', 'A20'] = '6:10-6:50'
```

```
    df_trn.loc[df_trn['样本 id'] == 'sample_1500', 'A20'] = '23:00-
23:30'

    df_trn.loc[df_trn['样本 id'] == 'sample_1524', 'A24'] = '15:00:00'
    df_trn.loc[df_trn['样本 id'] == 'sample_1524', 'A26'] = '15:30:00'

    df_trn.loc[df_trn['样本 id'] == 'sample_1046', 'A28'] = '18:00-
18:30'

    df_trn.loc[df_trn['样本 id'] == 'sample_1230', 'B5'] = '17:00:00'
    df_trn.loc[df_trn['样本 id'] == 'sample_97', 'B7'] = '1:00:00'
    df_trn.loc[df_trn['样本 id'] == 'sample_752', 'B9'] = '11:00-14:00'

    df_trn.loc[df_trn['样本 id'] == 'sample_609', 'B11'] = '11:00-12:00'
    df_trn.loc[df_trn['样本 id'] == 'sample_643', 'B11'] = '12:00-13:00'
    df_trn.loc[df_trn['样本 id'] == 'sample_1164', 'B11'] = '5:00-6:00'
    return df_trn


def test_a_abnormal_adjust(data):
    df_tst = data.copy()
    df_tst.loc[df_tst['样本 id'] == 'sample_919', 'A9'] = '19:50:00'
    return df_tst


def test_b_abnormal_adjust(data):
    df_tst = data.copy()
    df_tst.loc[df_tst['样本 id'] == 'sample_566', 'A5'] = '18:00:00'
    df_tst.loc[df_tst['样本 id'] == 'sample_40', 'A20'] = '5:00-5:30'
    df_tst.loc[df_tst['样本 id'] == 'sample_531', 'B5'] = '1:00'
    return df_tst
---


+*In[8]:*+
[source, ipython3]
---
"""开始修正"""
df_trn = train_abnormal_revise(df_trn).pipe(test_a_abnormal_adjust)
df_tst_a                                                          =
test_a_abnormal_revise(df_tst_a).pipe(test_a_abnormal_revise)   # 这个
pipe 就相当于连着处理
df_tst_b = test_b_abnormal_adjust(df_tst_b)
---
```

=== 标签与数据集整合

+*In[9]:*+
[source, ipython3]
----
df_trn, df_tst = df_trn.copy(), df_tst_a.copy()
df_target = df_trn['收率'] # 单独把收率拿出来
del df_trn['收率']

下面也是类似于一个数据处理的技巧，就是训练集和测试集放在一块预处理操作会比较方便
df_trn_tst = df_trn.append(df_tst, ignore_index=False).reset_index(drop=True)

=== 某些列的缺失值处理

对于缺失值的处理，可以使用众数，中位数，平均值等进行填充，

+*In[10]:*+
[source, ipython3]
----
df_trn['A3'].isnull().sum() 42

A3 列有缺失值，所以可以进行一个填充，使用众数
for _df in [df_trn, df_tst, df_trn_tst]:
    _df['A3'] = _df['A3'].fillna(405)

== 特征工程

这一部分，要提取特征出来，虽然我们收到的数据是结构化好了的数据，但是大部分时候，这些数据不能直接用，第一会有冗余性，第二特征不足，模型可能无法学习，所以我们需要特征工程部分的处理，这一部分很重要，数据和特征决定了机器学习的上限，而模型和算法只是去逼近这个上限而已，所以模型的好坏直接取决于特征工程部分。
这一块主要可以分为下面几部分：* 时间段和时间特征预处理 * 温度相关特征，温度相关统计特征 * 时间相关特征 * 水耗相关特征 * 所有特征进行合并

=== 时间和时间段特征预处理

这一块，主要是针对时间特征的一个处理，首先把所有时间相关的列拿出来，然后同时对测试集，训练集进行处理，对于每一列时间特征，我们先改名字（带上时间），这样后面处理的时候方便查找操作，对于时间段的话，可以进行拆分成起止时间和终止时间，也可以求出持续时间，换成三个特征。
这些都是 tricks。对于拆分好的时间，我们转换成时长，这样计算机才认识。

+*In[11]:*+
[source, ipython3]

```
# 所有时间相关列
cols_timer = ['A5', 'A7', 'A9', 'A11', 'A14', 'A16', 'A24', 'A26', 'B5', 'B7']

# 同时对训练和测试集进行相同处理，不管用得上用不上，先处理完再说
for _df in [df_trn_tst, df_trn, df_tst]:
    # 添加列名标记
    _df.rename(columns={_col:_col+'_t' for _col in cols_timer}, inplace=True)
    # 遍历所有持续时间相关列  如 21:00-21:30
    for _col in ['A20', 'A28', 'B4', 'B9', 'B10', 'B11']:
        # 获取当前列的索引
        _idx_col = _df.columns.tolist().index(_col)
        # 添加新的一列，表示起始时间， split 表示分别取开始和结束时间，用索引来指定
        _df.insert(_idx_col+1,    _col+'_at',    _df[_col].str.split('-').str[0])
        #添加新的一列，表示终止时间
        _df.insert(_idx_col+2,    _col+'_bt',    _df[_col].str.split('-').str[1])

        # 删除持续时间
        del _df[_col]
        cols_timer = cols_timer + [_col + '_at', _col+'_bt']
```

+*In[12]:*+
[source, ipython3]

```
"""查看一下效果"""
df_trn_tst.head()
```

+*Out[12]:*+

[cols=",,,,,,,,,,,,,,,,,,,,,",options="header",]
|===
==
| |样本 id |A1 |A2 |A3 |A4 |A5_t |A6 |A7_t |A8 |A9_t |... |B8 |B9_at
|B9_bt |B10_at |B10_bt |B11_at |B11_bt |B12 |B13 |B14
|0 |sample_1528 |300 |NaN |405.0 |700 |13:30:00 |38.0 |NaN |NaN
|15:30:00 |... |45.0 |11:30 |13:00 |14:00 |15:30 |NaN |NaN |800.0 |0.15
|400

|1 |sample_1698 |300 |NaN |405.0 |700 |14:00:00 |29.0 |NaN |NaN
|16:00:00 |... |45.0 |6:00 |7:30 |7:30 |9:00 |9:00 |10:00 |1200.0 |0.15
|400

|2 |sample_639 |300 |NaN |405.0 |700 |14:00:00 |29.0 |NaN |NaN |16:00:00
|... |45.0 |1:00 |2:30 |2:30 |4:00 |4:00 |5:00 |1200.0 |0.15 |400

|3 |sample_483 |300 |NaN |405.0 |700 |1:30:00 |38.0 |NaN |NaN |3:00:00
|... |45.0 |19:00 |20:30 |21:30 |23:00 |NaN |NaN |800.0 |0.15 |400

|4 |sample_617 |300 |NaN |405.0 |700 |22:00:00 |29.0 |NaN |NaN |0:00:00
|... |45.0 |9:00 |10:30 |10:30 |12:00 |12:00 |13:00 |1200.0 |0.15 |420
|===
==

5 rows × 49 columns

+*In[13]:*+
[source, ipython3]

"""我们把带有时间特征的列筛选出来"""
cols_timer = list(filter(lambda x: x.endswith('t'), df_trn_tst.columns))
cols_timer

+*Out[13]:*+
----['A5_t',
 'A7_t',
 'A9_t',

```
 'A11_t',
 'A14_t',
 'A16_t',
 'A20_at',
 'A20_bt',
 'A24_t',
 'A26_t',
 'A28_at',
 'A28_bt',
 'B4_at',
 'B4_bt',
 'B5_t',
 'B7_t',
 'B9_at',
 'B9_bt',
 'B10_at',
 'B10_bt',
 'B11_at',
 'B11_bt']----


+*In[14]:*+
[source, ipython3]
----
"""将时间全部全换成分钟的形式"""
def time_to_min(x):
    if x is np.nan:
        return np.nan
    else:
        x = x.replace(';', ':').replace('; ', ':')
        x = x.replace('::', ':').replace('"', ':')
        h, m = x.split(':')[:2]
        h = 0 if not h else h
        m = 0 if not m else m
        return int(h)*60 + int(m)
----


+*In[15]:*+
[source, ipython3]
----
for col in cols_timer:
    print(col, ": ", df_trn_tst[col].value_counts().index)
----
```

+*Out[15]:*+

A5_t : Index(['6:00:00', '14:00:00', '22:00:00', '21:00:00', '8:00:00',
'18:00:00',
'1:00:00', '2:00:00', '17:00:00', '12:30:00', '11:00:00',
'12:00:00',
'15:00:00', '7:00:00', '5:00:00', '23:00:00', '0:00:00',
'4:00:00',
'16:00:00', '9:00:00', '21:30:00', '13:00:00', '10:00:00',
'6:30:00',
'15:30:00', '19:00:00', '3:30:00', '20:00:00', '3:00:00',
'7:30:00',
'17:30:00', '16:30:00', '10:30:00', '14:30:00', '22:30:00',
'2:30:00',
'13:30:00', '1:30:00', '20:30:00', '8:30:00', '4:30:00',
'5:30:00',
'18:30:00', '19:30:00', '9:30:00', '11:30:00', '13:50:00',
'10:45:00',
'15:40:00', '23:30:00', '0:30:00', '14:50:00', '10:40:00',
'8:15:00',
'16:40:00', '6:50:00', '8:45:00', '2:45:00', '13:10:00',
'15:10:00',
'7:45:00', '11:45:00', '3:45:00', '11:20:00', '9:40:00'],
dtype='object')

A7_t : Index(['12:40:00', '2:10:00', '15:40:00', '7:00:00', '7:10:00',
'7:40:00',
'6:15:00', '21:45:00', '21:40:00', '2:30:00', '2:45:00',
'15:20:00',
'12:15:00', '12:30:00', '19:40:00', '17:10:00', '17:30:00',
'23:30:00',
'21:20:00', '22:00:00', '13:10:00', '4:30:00', '13:40:00',
'6:20:00',
'15:10:00', '7:45:00', '15:45:00', '9:30:00', '2:40:00',
'6:40:00',
'3:00:00', '22:30:00', '12:45:00', '21:15:00', '4:20:00',
'13:00:00',
'20:40:00', '16:20:00', '18:10:00', '16:00:00', '3:30:00',
'21:10:00',
'21:30:00', '5:30:00', '1:50:00', '8:40:00', '1:10:00',
'21:50:00',
'14:40:00', '2:20:00', '6:30:00', '21:00:00', '7:30:00',
'8:10:00',

```
       '22:20:00',  '11:15:00',  '17:15:00',  '16:40:00',  '1:30:00',
'22:40:00',
       '13:45:00',  '13:15:00',  '17:45:00',  '20:15:00',  '18:00:00',
'12:50:00',
       '11:45:00',  '12:00:00',  '3:50:00',  '8:15:00',  '9:00:00',
'5:15:00',
       '14:15:00',  '6:25:00',  '1:00:00',  '19:30:00',  '3:40:00',
'15:15:00',
       '15:00:00'],
      dtype='object')
A9_t :  Index(['8:00:00', '16:00:00', '0:00:00', '23:00:00', '3:00:00',
'23:30:00',
       '20:00:00',  '19:00:00',  '11:00:00',  '10:00:00',  '15:00:00',
'5:00:00',
       '14:00:00',  '17:00:00',  '7:00:00',  '12:30:00',  '7:30:00',
'2:30:00',
       '9:00:00',  '15:30:00',  '13:30:00',  '4:00:00',  '22:30:00',
'6:30:00',
       '17:30:00',  '14:30:00',  '18:00:00',  '6:00:00',  '18:30:00',
'9:30:00',
       '3:30:00',  '8:30:00',  '1:30:00',  '22:00:00',  '16:30:00',
'11:30:00',
       '21:00:00',  '0:30:00',  '13:00:00',  '4:30:00',  '10:30:00',
'12:00:00',
       '19:30:00',  '1:00:00',  '5:30:00',  '20:30:00',  '2:00:00',
'14:20:00',
       '21:30:00',  '8:20:00',  '23:20:00',  '8:15:00',  '18:40:00',
'7:40:00',
       '14:40:00',  '17:15:00',  '8:50:00',  '11:50:00',  '9:40:00',
'5:45:00',
       '9:20:00',  '4:40:00',  '16:50:00',  '23:40:00',  '19:50:00',
'23:15:00',
       '14:10:00',  '13:40:00',  '16:20:00',  '8:45:00',  '8:05:00',
'6:50:00',
       '22:20:00',  '15:40:00',  '13:20:00',  '18:10:00',  '16:15:00',
'10:35:00',
       '7:10:00',  '4:15:00',  '13:50:00',  '22:45:00',  '17:50:00',
'2:40:00',
       '3:10:00',  '9:10:00',  '22:50:00',  '10:45:00',  '14:15:00',
'17:10:00',
       '21:20:00',  '14:50:00',  '18:05:00',  '10:10:00',  '17:20:00',
'15:20:00'],
      dtype='object')
All_t :  Index(['9:00:00', '17:00:00', '1:00:00', '0:00:00', '4:00:00',
```

'21:00:00',
       '20:00:00', '0:30:00', '11:00:00', '16:00:00', '12:00:00',
'6:00:00',
       '15:00:00', '8:00:00', '13:30:00', '8:30:00', '16:30:00',
'3:30:00',
       '23:30:00', '14:30:00', '7:30:00', '18:00:00', '18:30:00',
'10:00:00',
       '5:00:00', '19:00:00', '2:30:00', '7:00:00', '17:30:00',
'9:30:00',
       '10:30:00', '15:30:00', '19:30:00', '4:30:00', '23:00:00',
'12:30:00',
       '22:00:00', '1:30:00', '13:00:00', '5:30:00', '14:00:00',
'11:30:00',
       '6:30:00', '2:00:00', '20:30:00', '3:00:00', '9:20:00',
'21:30:00',
       '15:20:00', '22:30:00', '0:20:00', '19:40:00', '9:15:00',
'15:40:00',
       '20:50:00', '9:50:00', '10:20:00', '6:45:00', '18:15:00',
'10:40:00',
       '8:40:00', '17:15:00', '7:50:00', '12:50:00', '11:10:00',
'14:40:00',
       '11:45:00', '19:10:00', '8:10:00', '5:40:00', '22:20:00',
'0:15:00',
       '23:50:00', '18:20:00', '10:10:00', '4:10:00', '18:10:00',
'23:20:00',
       '16:40:00', '14:50:00', '16:20:00', '5:15:00', '0:40:00',
'15:10:00',
       '17:50:00', '3:40:00', '15:50:00', '12:15:00', '19:15:00',
'00:30:00',
       '15:15:00', '23:45:00', '18:50:00', '9:05:00', '11:35:00',
'14:20:00'],
      dtype='object')
A14_t : Index(['10:00:00', '18:00:00', '2:00:00', '1:00:00', '5:00:00',
'22:00:00',
       '1:30:00', '21:00:00', '12:00:00', '13:00:00', '17:00:00',
'7:00:00',
       '16:00:00', '14:30:00', '17:30:00', '9:00:00', '4:30:00',
'8:30:00',
       '0:30:00', '15:30:00', '9:30:00', '11:00:00', '19:00:00',
'6:00:00',
       '19:30:00', '20:00:00', '16:30:00', '11:30:00', '20:30:00',
'18:30:00',
       '5:30:00', '8:00:00', '3:30:00', '10:30:00', '0:00:00',
'13:30:00',

'2:30:00', '23:00:00', '14:00:00', '15:00:00', '6:30:00',
'12:30:00',
'3:00:00', '21:30:00', '7:30:00', '22:30:00', '10:20:00',
'4:00:00',
'16:20:00', '23:30:00', '1:20:00', '20:40:00', '10:15:00',
'19:15:00',
'7:45:00', '9:40:00', '10:50:00', '9:10:00', '21:50:00',
'11:40:00',
'16:40:00', '11:20:00', '11:10:00', '10:05:00', '6:40:00',
'19:10:00',
'13:50:00', '6:15:00', '17:20:00', '16:15:00', '16:10:00',
'23:20:00',
'17:40:00', '12:45:00', '19:50:00', '12:10:00', '15:40:00',
'12:35:00',
'0:50:00', '5:10:00', '20:10:00', '16:50:00', '13:15:00',
'20:15:00',
'19:20:00', '0:45:00', '18:50:00', '8:50:00', '1:40:00',
'1:15:00',
'4:40:00', '0:20:00', '18:15:00', '15:50:00', '15:20:00'],
dtype='object')
A16_t : Index(['11:00:00', '19:00:00', '3:00:00', '2:00:00', '6:00:00',
'23:00:00',
'2:30:00', '22:00:00', '13:00:00', '14:00:00', '18:00:00',
'17:00:00',
'8:00:00', '5:30:00', '15:30:00', '18:30:00', '10:00:00',
'10:30:00',
'12:00:00', '20:30:00', '16:30:00', '9:30:00', '7:00:00',
'1:30:00',
'20:00:00', '21:00:00', '17:30:00', '21:30:00', '12:30:00',
'11:30:00',
'19:30:00', '9:00:00', '1:00:00', '6:30:00', '4:30:00',
'3:30:00',
'14:30:00', '0:00:00', '15:00:00', '13:30:00', '22:30:00',
'16:00:00',
'7:30:00', '4:00:00', '8:30:00', '23:30:00', '5:00:00',
'11:20:00',
'0:30:00', '17:20:00', '21:40:00', '11:15:00', '12:20:00',
'8:45:00',
'20:15:00', '2:20:00', '11:50:00', '10:40:00', '22:50:00',
'17:40:00',
'12:40:00', '7:40:00', '11:05:00', '5:40:00', '18:20:00',
'7:15:00',
'17:15:00', '0:20:00', '2:40:00', '10:10:00', '9:50:00',
'20:50:00',

```
       '21:15:00',  '13:10:00',  '13:45:00',  '2:15:00',  '13:35:00',
'19:15:00',
       '20:10:00',  '1:10:00',  '17:50:00',  '19:50:00',  '1:45:00',
'1:20:00',
       '6:10:00',  '16:50:00',  '14:50:00',  '14:15:00',  '16:20:00',
'16:40:00',
       '12:10:00',  '1:50:00',  '21:10:00',  '18:40:00',  '17:10:00',
'20:20:00'],
      dtype='object')
A20_at :  Index(['11:00', '19:00', '3:00', '2:00', '6:00', '23:00',
'11:30', '16:00',
       '13:00', '6:30', '14:00', '20:00', '1:00', '12:00', '20:30',
'0:00',
       '14:30', '0:30', '4:00', '10:00', '2:30', '15:00', '21:00',
'17:00',
       '15:30', '22:00', '18:00', '4:30', '5:00', '1:30', '7:00',
'19:30',
       '3:30', '17:30', '12:30', '18:30', '16:30', '8:30', '22:10',
'9:30',
       '8:00', '18:10', '10:10', '21:30', '10:30', '14:10', '13:30',
'22:30',
       '9:00', '5:30', '6:10', '2:10', '7:30', '18:00:', '12:40', '1;30',
       '12:10', '19:10', '10:40', '23:30', '1:20', '2:40', '14:40',
'13:10',
       '16:40', '19:15', '23:40', '17:20', '22:40', '9:40', '15:10',
'14:50',
       '5;30', '7:20', '0:20', '2:0', '3:20', '23:15', '12:20', '9:20',
       '12:45', '14:0', '11:50', '11:10', '23:20', '10:45', '18:20',
'8:45',
       '14:20', '2:20', '15:20', '16:10', '3:10', '5:20', '10:20',
'4:40',
       '11;30', '5:50', '11:20', '14:15'],
      dtype='object')
A20_bt :  Index(['12:00', '20:00', '4:00', '3:00', '7:00', '16:30',
'6:30', '0:00',
       '1:00', '14:00', '15:00', '14:30', '20:30', '10:30', '21:00',
'5:00',
       '12:30', '3:30', '16:00', '2:00', '11:30', '4:30', '19:30',
'1:30',
       '23:30', '15:30', '17:30', '7:30', '18:00', '21:30', '2:30',
'0:30',
       '5:30', '17:00', '19:00', '22:30', '18:30', '13:00', '22:00',
'23:00',
       '13:30', '22:40', '9:00', '8:00', '18:40', '10:40', '14:40',
```

```
'6:00',
       '10:00', '6:40', '9:30', '11:00', '2:40', '8:30', '11:0', '7:0',
       '12:20', '12:50', '1:50', '2;00', '18;30', '9:50', '11:15',
'3:50',
       '16;30', '0:50', '17:10', '10:50', '14:50', '7:50', '15:50',
'17;00',
       '20:50', '4:10', '0:10', '15:10', '11:50', '5:50', '3:40',
'18:50',
       '23:50', '17:50', '2:50', '23:35', '13:40', '11:40', '6;00',
'13:45',
       '9:45', '13:10', '23:10', '19:40', '13:15'],
      dtype='object')
A24_t :  Index(['12:00:00', '20:00:00', '4:00:00', '3:00:00', '7:00:00',
'0:00:00',
       '12:30:00', '14:00:00', '15:00:00', '17:00:00', '19:00:00',
'1:30:00',
       '1:00:00', '23:00:00', '7:30:00', '21:00:00', '5:00:00',
'15:30:00',
       '11:00:00', '16:30:00', '16:00:00', '18:00:00', '11:30:00',
'6:00:00',
       '2:00:00', '5:30:00', '22:00:00', '14:30:00', '6:30:00',
'20:30:00',
       '13:00:00', '18:30:00', '4:30:00', '3:30:00', '2:30:00',
'23:30:00',
       '19:30:00', '21:30:00', '13:30:00', '8:00:00', '0:30:00',
'17:30:00',
       '10:30:00', '10:00:00', '22:30:00', '9:00:00', '9:30:00',
'12:20:00',
       '11:10:00', '20:10:00', '19:40:00', '20:15:00', '7:20:00',
'7:40:00',
       '9:45:00', '10:20:00', '2:10:00', '0:20:00', '14:50:00',
'13:20:00',
       '8:20:00', '16:40:00', '14:40:00', '13:10:00', '12:50:00',
'17:20:00',
       '11:05:00', '11:15:00', '3:10:00', '18:20:00', '17:25:00',
'21:20:00',
       '11:20:00', '3:45:00', '1:10:00', '12:40:00', '0:40:00',
'14:10:00',
       '8:30:00', '11:40:00', '22:40:00', '3:50:00', '5:10:00',
'23:45:00',
       '14:15:00', '2:20:00', '16:20:00', '6:20:00', '15:10:00',
'19:20:00',
       '1:20:00', '21:10:00', '15:20:00'],
      dtype='object')
```

A26_t : Index(['13:00:00', '21:00:00', '5:00:00', '4:00:00', '8:00:00',
'15:00:00',
'7:30:00', '1:00:00', '2:00:00', '16:00:00', '12:30:00',
'17:00:00',
'17:30:00', '5:30:00', '23:30:00', '11:30:00', '19:30:00',
'4:30:00',
'16:30:00', '15:30:00', '3:30:00', '12:00:00', '7:00:00',
'1:30:00',
'3:00:00', '0:30:00', '21:30:00', '18:30:00', '19:00:00',
'20:00:00',
'6:00:00', '22:30:00', '20:30:00', '0:00:00', '6:30:00',
'22:00:00',
'13:30:00', '18:00:00', '2:30:00', '14:00:00', '23:00:00',
'8:30:00',
'14:30:00', '11:00:00', '10:00:00', '10:30:00', '9:30:00',
'9:00:00',
'11:40:00', '4:50:00', '5:40:00', '12:10:00', '17:10:00',
'12:50:00',
'11:35:00', '17:55:00', '21:40:00', '13:40:00', '18:50:00',
'15:50:00',
'15:20:00', '8:50:00', '17:50:00', '1:40:00', '14:55:00',
'21:50:00',
'15:01:00', '10:15:00', '13:50:00', '21:45:00', '4:45:00',
'15:40:00',
'13:20:00', '0:15:00', '11:50:00', '20:40:00', '1:50:00',
'3:40:00',
'6:50:00', '14:40:00', '16:50:00', '11:45:00', '10:50:00',
'0:50:00',
'1:10:00', '20:10:00', '2:50:00', '7:50:00', '19:50:00'],
dtype='object')
A28_at : Index(['13:00', '21:00', '5:00', '4:00', '19:30', '11:00',
'1:00', '15:00',
'10:30', '23:30', '14:00', '8:00', '15:30', '3:00', '0:00',
'3:30',
'16:00', '11:30', '23:00', '18:00', '5:30', '7:00', '6:00',
'17:00',
'19:00', '4:30', '16:30', '22:30', '20:00', '22:00', '14:30',
'0:30',
'12:00', '2:00', '7:30', '21:30', '10:00', '6:30', '18:30',
'13:30',
'2:30', '17:30', '12:30', '20:30', '1:30', '9:00', '13；00',
'8:30',
'16:10', '12:50', '14:15', '9:30', '8:50', '15:10', '3:40',
'15:40',

'15:50', '9:10', '2:50', '21:15', '17:55', '1:10', '10:15', '0:50',
'14:10', '17:40', '14:55', '21:50', '7:50', '20:10', '10:40', '16:50',
'19:50', '0:10', '9:40', '11:10', '14::30', '4:45', '9:20', '17:50',
'11:50', '18:50', '11:35', '10:50', '6:50', '13:20', '1:50', '4:40',
'1:40', '15:20', '13:50', '0:15', '14；00', '10:10', '21:40'],
dtype='object')
A28_bt : Index(['14:00', '22:00', '6:00', '5:00', '16:00', '20:00', '11:30', '0:30',
'9:00', '6:30', '2:00', '16:30', '0:00', '11:00', '4:30', '20:30',
'4:00', '15:00', '19:00', '12:00', '18:00', '1:00', '1:30', '12:30',
'8:00', '17:00', '14:30', '5:30', '23:30', '17:30', '3:30', '23:00',
'13:00', '15:30', '22:30', '8:30', '3:00', '7:00', '13:30', '19:30',
'18:30', '2:30', '21:00', '7:30', '10:00', '10:30', '21:30', '9:30',
'22:40', '14:20', '5:45', '14:50', '15:40', '24:00', '22；00', '9:10',
'17:15', '14:40', '11:15', '18:25', '9:40', '16:40', '10:10', '15:35',
'0:0', '11:40', '6:0', '11:10', '13:20', '1:50', '0:45', '6:40',
'16:10', '0:55', '17:10', '7:40', '15:15', '8:40'],
dtype='object')
B4_at : Index(['14:00', '22:00', '6:00', '5:00', '16:00', '20:00', '11:30', '9:00',
'1:00', '0:00', '2:00', '6:30', '11:00', '4:00', '16:30', '15:00',
'12:00', '1:30', '4:30', '19:00', '14:30', '18:00', '8:00', '0:30',
'20:30', '17:30', '5:30', '13:00', '17:00', '12:30', '23:00', '3:30',
'21:00', '15:30', '23:30', '3:00', '7:00', '22:30', '8:30', '13:30',
'18:30', '2:30', '19:30', '7:30', '9:30', '21:30', '10:30', '10:00',
'15:0', '14:20', '22:40', '24:00', '15:40', '14:50', '5:45', '19:05',
'10:15', '9:10', '17:15', '11:15', '12；30', '17:0', '20；00', '15:35',
'0；00', '22″00', '1:25', '20;30', '1:50', '11;30', '6:40',

```
'16:10',
       '0:55', '17:10', '15；00', '8:40'],
      dtype='object')
B4_bt :  Index(['15:00', '23:00', '7:00', '6:00', '3:00', '13:00',
'17:00', '10:00',
       '16:30', '2:00', '19:00', '18:00', '1:30', '22:00', '14:00',
'13:30',
       '12:30', '8:00', '7:30', '5:00', '5:30', '21:30', '15:30', '2:30',
       '17:30', '6:30', '20:30', '0:00', '0:30', '18:30', '21:00',
'9:00',
       '16:00', '1:00', '9:30', '20:00', '4:30', '22:30', '4:00',
'23:30',
       '19:30', '11:00', '3:30', '8:30', '14:30', '12:00', '22;00',
'10:30',
       '11:30', '7:50', '17;30', '3;00', '18;00', '14;00', '15:20',
'15:50',
       '8；00', '15:0', '18:20', '12:15', '9:40', '23:40', '20:05', '22；
00',
       '2:50', '17:05', '17:002', '13；00', '21:0', '7:40', '10:10',
'1:55'],
      dtype='object')
B5_t :  Index(['15:00:00', '23:00:00', '7:00:00', '6:00:00', '3:00:00',
'13:00:00',
       '17:00:00', '10:00:00', '22:00:00', '16:30:00', '18:00:00',
'2:00:00',
       '1:30:00', '8:00:00', '13:30:00', '19:00:00', '12:30:00',
'14:00:00',
       '5:30:00', '17:30:00', '15:30:00', '7:30:00', '5:00:00',
'21:30:00',
       '2:30:00', '0:00:00', '6:30:00', '20:00:00', '18:30:00',
'21:00:00',
       '0:30:00', '20:30:00', '9:30:00', '9:00:00', '1:00:00',
'4:30:00',
       '22:30:00', '16:00:00', '4:00:00', '11:00:00', '12:00:00',
'3:30:00',
       '23:30:00', '14:30:00', '19:30:00', '8:30:00', '11:30:00',
'10:30:00',
       '7:50:00', '7:40:00', '15:50:00', '1:55:00', '12:15:00',
'2:50:00',
       '23:40:00', '9:40:00', '20:05:00', '22:20:00', '17:05:00',
'10:10:00',
       '18:20:00', '15:20:00'],
      dtype='object')
B7_t :  Index(['17:00:00', '1:00:00', '9:00:00', '8:00:00', '19:00:00',
```

'12:00:00',
       '4:00:00', '21:00:00', '5:00:00', '3:00:00', '16:00:00',
'23:00:00',
       '10:00:00', '0:00:00', '19:30:00', '22:00:00', '6:00:00',
'20:00:00',
       '10:30:00', '3:30:00', '15:00:00', '11:00:00', '20:30:00',
'15:30:00',
       '21:30:00', '14:00:00', '13:00:00', '7:30:00', '23:30:00',
'11:30:00',
       '2:30:00', '16:30:00', '2:00:00', '7:00:00', '22:30:00',
'14:30:00',
       '18:00:00', '4:30:00', '17:30:00', '5:30:00', '8:30:00',
'6:30:00',
       '1:30:00', '0:30:00', '9:30:00', '12:30:00', '18:30:00',
'13:30:00',
       '17:40:00', '22:45:00', '17:10:00', '8:15:00', '1:20:00',
'3:50:00',
       '17:05:00', '13:20:00', '7:20:00', '17:20:00', '14:20:00'],
      dtype='object')
B9_at : Index(['17:00', '1:00', '9:00', '8:00', '19:00', '12:00',
'5:00', '3:00',
       '21:00', '12:30', '0:00', '20:00', '20:30', '14:00', '6:00',
'10:30',
       '22:00', '16:00', '23:00', '16:30', '19:30', '4:00', '8:30',
'23:30',
       '10:00', '9:30', '3:30', '13:30', '14:30', '2:00', '4:30',
'21:30',
       '15:00', '15:30', '2:30', '13:00', '0:30', '11:30', '1:30',
'4:10',
       '17:30', '12:10', '11:00', '7:00', '5:30', '22:30', '0:10',
'7:30',
       '6:30', '8:10', '16:10', '18:30', '23:40', '3:40', '20:10',
'22:40',
       '18;00', '16;30', '20:40', '1；00', '15:40', '16:20', '11:10',
'7:20',
       '8:0', '17:20', '19:20', '7:35', '18:00', '17:45', '6:10', '0:50',
       '16；00', '14:20', '19:40', '3;30', '3；00', '17:10', '13:40',
'17；00',
       '8:20', '7:40'],
      dtype='object')
B9_bt : Index(['18:30', '10:30', '2:30', '21:00', '5:00', '12:00',
'9:30', '20:30',
       '11:00', '13:30', '4:30', '6:30', '22:30', '14:00', '23:30',
'1:30',

```
       '15:30', '22:00', '17:30', '10:00', '21:30', '11:30', '5:30',
'18:00',
       '4:00', '0:30', '16:00', '7:30', '17:00', '6:00', '14:30', '1:00',
       '23:00', '19:30', '2:00', '19:00', '16:30', '15:00', '13:00',
'8:00',
       '7:00', '3:30', '3:00', '9:00', '12:30', '0:00', '1:20', '20:00',
       '9:20', '5:20', '16:50', '8:30', '4:50', '0:50', '13:20', '12:50',
       '22:0', '8:50', '13:40', '9:10', '10:10', '15;00', '18:40',
'19:20',
       '4::00', '1:10', '17:20', '00:30', '17:10', '15:10', '18:20',
'20:45',
       '23:0', '7:40', '21:10', '20:50', '8:35', '5:50', '12:20',
'13:10',
       '21:20', '0:10', '18;30', '10:40', '8:40'],
      dtype='object')
B10_at :  Index(['10:30', '18:30', '2:30', '10:00', '13:30', '6:30',
'23:00', '21:30',
       '20:30', '19:00', '5:30', '5:00', '14:00', '11:30', '14:30',
'16:30',
       '18:00', '7:30', '9:30', '22:30', '19:30', '7:00', '0:30', '1:30',
       '3:00', '13:00', '23:30', '15:30', '15:00', '22:00', '17:30',
'4:30',
       '0:00', '11:00', '17:00', '3:30', '16:00', '6:00', '21:00',
'1:00',
       '2:00', '8:00', '4:00', '20:00', '19:0', '12:30', '9:00', '8:30',
       '17:40', '12:00', '5:40', '1:40', '1:10', '9:10', '17:10', '9:40',
       '21:10', '21:40', '19;00', '10:0', '1:20', '13:10', '5:10',
'13:40',
       '19:20', '5:15', '3:20', '18::30', '2:10', '3:10', '22:20',
'18:40',
       '16:40', '21:20', '3:40', '22:10', '15:40', '20:45', '13::30',
'14;00',
       '13:20', '5:20', '10; 00', '6:40', '14:35', '0:50', '16:10',
'13:50',
       '12:40', '10:10', '8:40', '23:0', '10:15'],
      dtype='object')
B10_bt :  Index(['12:00', '20:00', '4:00', '11:30', '15:00', '8:00',
'23:00', '7:00',
       '21:00', '22:00', '18:00', '6:30', '0:30', '1:00', '16:00',
'20:30',
       '0:00', '15:30', '13:00', '9:00', '17:00', '11:00', '19:30',
'19:00',
       '2:00', '12:30', '3:00', '14:30', '2:30', '1:30', '7:30', '4:30',
       '10:30', '22:30', '6:00', '18:30', '5:00', '5:30', '16:30',
```

```
'8:30',
       '10:00', '23:30', '14:00', '9:30', '3:30', '21:30', '17:30',
'14:50',
       '2:50', '20;30', '6:50', '13:30', '2:45', '6:40', '22:40',
'10:40',
       '10:50', '18:50', '23:50', '22:50', '6；30', '22:15', '16;00',
'18:40',
       '19:10', '17:10', '10:20', '5:10', '11:45', '23:20', '17:40',
'4:20',
       '13:10', '8:45', '20:10', '20:20', '14:0', '23:10', '3:10',
'2:40',
       '2:20', '1:0', '14;30', '15;30', '15:10', '18:20', '8:20',
'21:40'],
      dtype='object')
Bll_at :  Index(['20:00', '12:00', '4:00', '22:00', '15:00', '8:00',
'9:00', '11:00',
       '21:00', '23:00', '17:00', '10:00', '13:00', '1:00', '3:00',
'14:00',
       '0:00', '10:30', '18:00', '19:30', '6:00', '5:00', '8:30', '0:30',
       '7:00', '20:10', '8:45', '21:20', '2:30'],
      dtype='object')
Bll_bt :  Index(['21:00', '13:00', '5:00', '23:00', '16:00', '9:00',
'12:00', '10:00',
       '21:30', '22:00', '5:30', '13:30', '18:00', '2:00', '0:00',
'10:30',
       '1:00', '4:00', '14:00', '11:00', '20:30', '15:00', '19:00',
'11:30',
       '0:30', '8:00', '7:00', '9:30', '22:20', '3:30'],
      dtype='object')
---
```

```
+*In[16]:*+
[source, ipython3]
---
for _df in [df_trn_tst, df_trn, df_tst]:
    for _col in cols_timer:
        _df[_col] = _df[_col].map(time_to_min)
---
```

```
+*In[17]:*+
[source, ipython3]
---
```

df_trn_tst.head()

+*Out[17]:*+

[cols=",,,,,,,,,,,,,,,,,,,,,",options="header",]
|===
==
| |样本 id |A1 |A2 |A3 |A4 |A5_t |A6 |A7_t |A8 |A9_t |... |B8 |B9_at
|B9_bt |B10_at |B10_bt |B11_at |B11_bt |B12 |B13 |B14
|0 |sample_1528 |300 |NaN |405.0 |700 |810 |38.0 |NaN |NaN |930 |...
|45.0 |690 |780 |840.0 |930.0 |NaN |NaN |800.0 |0.15 |400

|1 |sample_1698 |300 |NaN |405.0 |700 |840 |29.0 |NaN |NaN |960 |...
|45.0 |360 |450 |450.0 |540.0 |540.0 |600.0 |1200.0 |0.15 |400

|2 |sample_639 |300 |NaN |405.0 |700 |840 |29.0 |NaN |NaN |960 |...
|45.0 |60 |150 |150.0 |240.0 |240.0 |300.0 |1200.0 |0.15 |400

|3 |sample_483 |300 |NaN |405.0 |700 |90 |38.0 |NaN |NaN |180 |... |45.0
|1140 |1230 |1290.0 |1380.0 |NaN |NaN |800.0 |0.15 |400

|4 |sample_617 |300 |NaN |405.0 |700 |1320 |29.0 |NaN |NaN |0 |... |45.0
|540 |630 |630.0 |720.0 |720.0 |780.0 |1200.0 |0.15 |420
|===
==

5 rows × 49 columns

=== 正式进行特征工程

这一块就是进行特征的组合，提取，转换等操作了，主要分为温度相关特征，时间相关特征和水耗相关特征的提取等。

==== 创建一个新的 df 来准备添加特征

之所以要创建一个新的 df，是因为我们不想在原来的数据集上进一步特征工程，这样拼接太大了，毕竟我们要从不同的角度构建特征出来，所以我们建立一个新的 df 之后，我们就可以把新造出来的特征先加到这个空的 df 上，然后我们把这个和原来的进行拼接就是最后的特征了，这也是做特征工程的一种 trick 吧，我们只需要统一的样本 id。

+*In[18]:*+
[source, ipython3]
----
raw = df_trn_tst.copy()
df = pd.DataFrame(raw['样本 id'])
df.head()

+*Out[18]:*+

[cols=",",options="header",]
|==============
| |样本 id
|0 |sample_1528
|1 |sample_1698
|2 |sample_639
|3 |sample_483
|4 |sample_617
|==============

==== 温度相关特征

温度特征这块，也是分为四个过程，加热，水解，脱色，结晶。
在每一个过程当中，我们首先提取原始的温度特征，然后再尝试进行一些加减组合等。特征工程中，我们需要发挥想象力，尽可能多的创造特征，不用先考虑哪些特征可能好，可能不好，先弥补这个广度，所以我们不仅只做一些简单的加减组合，我们还要提取温度的统计特征。

+*In[19]:*+
[source, ipython3]

加热过程
df['P1_S1_A6_0C'] = raw['A6'] # 容器初始温度
df['P1_S2_A8_1C'] = raw['A8'] # 首次测温温度
df['P1_S3_A10_2C'] = raw['A10'] # 准备水解温度
df['P1_C1_C0_D'] = raw['A8'] - raw['A6'] # 测温温差
df['P1_C2_C0_D'] = raw['A10'] - raw['A6'] # 初次沸腾温差

水解过程
df['P2_S1_A12_3C'] = raw['A12'] # 水解开始温度

```
df['P2_S2_A15_4C'] = raw['A15']  # 水解过程测温温度
df['P2_S3_A17_5C'] = raw['A17'] # 水解结束温度
df['P2_C3_C0_D'] = raw['A12'] - raw['A6']  # 水解开始与初始温度温差
df['P2_C3_C2_D'] = raw['A12'] - raw['A10']  # 水解开始前恒温温差
df['P2_C4_C3_D'] = raw['A15'] - raw['A12']  # 水解过程中途温差
df['P2_C5_C4_D'] = raw['A17'] - raw['A15']  # 水解结束中途温差
df['P2_C5_C3_KD'] = raw['A17'] - raw['A12']  # 水解起止温差

# 脱色过程
df['P3_S2_A25_7C'] = raw['A25']  # 脱色保温开始温度
df['P3_S3_A27_8C'] = raw['A27']  # 脱色保温结束温度
df['P3_C7_C5_D'] = raw['A25'] - raw['A17']  # 降温温差
df['P3_C8_C7_KD'] = raw['A27'] - raw['A25']  # 保温温差

# 结晶过程
df['P4_S2_B6_11C'] = raw['B6']  # 结晶开始温度
df['P4_S3_B8_12C'] = raw['B8']  # 结晶结束温度
df['P4_C11_C8_D'] = raw['B6'] - raw['A27']  # 脱色结束到结晶温差
df['P4_C12_C11_KD'] = raw['B8'] - raw['B6']  # 结晶温差
```

温度相关的统计特征

+*In[20]:*+
[source, ipython3]
```
_funcs = ['mean', 'std', 'sum']
# 遍历每一种统计指标
for _func in _funcs:
    # 对每一个样本计算各项指标
    df[f'P2_C2-C5_{_func}']    =    raw[['A10',    'A12',    'A15',
'A17']].agg(_func, axis=1)  # 沸腾过程温度
    df[f'P2_D3-D5_{_func}'] = df[[f'P2_C{i}_C{i-1}_D' for i in range(3,
6)]].abs().agg(_func, axis=1) # 沸腾过程绝对温差
    df[f'P2_C1-C12_KD_ABS_{_func}'] = df[[_f for _f in df.columns if
_f.endswith('KD')]].abs().agg(_func, axis=1)  # 关键过程绝对温差
    df[f'P2_C1-C12_D_{_func}']  =  df[[_f  for  _f  in  df.columns  if
_f.endswith('D')]].abs().agg(_func, axis=1)  # 所有过程绝对温差
    df[f'P2_LARGE_KD_{_func}']  =  df[['P2_C3_C0_D',  'P3_C7_C5_D',
'P4_C12_C11_KD']].abs().agg(_func, axis=1)  # 大温差绝对温差
```

+*In[21]:*+
[source, ipython3]
----
df.head()

+*Out[21]:*+

[cols=",,,,,,,,,,,,,,,,,,,,,,",options="header",]
|===
==
| |样本 id |P1_S1_A6_OC |P1_S2_A8_1C |P1_S3_A10_2C |P1_C1_C0_D
|P1_C2_C0_D |P2_S1_A12_3C |P2_S2_A15_4C |P2_S3_A17_5C |P2_C3_C0_D |...
|P2_C2-C5_std |P2_D3-D5_std |P2_C1-C12_KD_ABS_std |P2_C1-C12_D_std
|P2_LARGE_KD_std |P2_C2-C5_sum |P2_D3-D5_sum |P2_C1-C12_KD_ABS_sum
|P2_C1-C12_D_sum |P2_LARGE_KD_sum
|0 |sample_1528 |38.0 |NaN |100 |NaN |62.0 |102.0 |103.0 |104.0 |64.0
|... |1.707825 |0.57735 |9.643651 |24.928565 |23.245071 |409.0 |4.0
|27.0 |191.0 |113.0

|1 |sample_1698 |29.0 |NaN |101 |NaN |72.0 |103.0 |104.0 |105.0 |74.0
|... |1.707825 |0.57735 |17.785762 |28.887521 |25.890796 |413.0 |4.0
|44.0 |226.0 |134.0

|2 |sample_639 |29.0 |NaN |102 |NaN |73.0 |103.0 |104.0 |105.0 |74.0
|... |1.290994 |0.00000 |18.009257 |29.231642 |25.514702 |414.0 |3.0
|43.0 |226.0 |135.0

|3 |sample_483 |38.0 |NaN |100 |NaN |62.0 |102.0 |103.0 |104.0 |64.0
|... |1.707825 |0.57735 |9.165151 |24.617293 |22.479620 |409.0 |4.0
|30.0 |207.0 |118.0

|4 |sample_617 |29.0 |NaN |101 |NaN |72.0 |103.0 |104.0 |105.0 |74.0
|... |1.707825 |0.57735 |17.785762 |28.887521 |25.890796 |413.0 |4.0
|44.0 |226.0 |134.0
|===
==

5 rows × 37 columns

+*In[22]:*+

[source, ipython3]

"""得到温度相关特征"""
df_temperature = df.set_index('样本 id')

+*In[23]:*+
[source, ipython3]
----
df_temperature.head()   # 这样后期利于拼接

+*Out[23]:*+

P1_S1_A6_0C

P1_S2_A8_1C

P1_S3_A10_2C

P1_C1_C0_D

P1_C2_C0_D

P2_S1_A12_3C

P2_S2_A15_4C

P2_S3_A17_5C

P2_C3_C0_D

P2_C3_C2_D

...

P2_C2-C5_std

P2_D3-D5_std

P2_C1-C12_KD_ABS_std

P2_C1-C12_D_std

P2_LARGE_KD_std

P2_C2-C5_sum

P2_D3-D5_sum

P2_C1-C12_KD_ABS_sum

P2_C1-C12_D_sum

P2_LARGE_KD_sum

样本 id

sample_1528

38.0

NaN

100

NaN

62.0

102.0

103.0

104.0

64.0

2.0

...

1.707825

0.57735

9.643651

24.928565

23.245071

409.0

4.0

27.0

191.0

113.0

sample_1698

29.0

NaN

101

NaN

72.0

103.0

104.0

105.0

74.0

2.0

...

1.707825

0.57735

17.785762

28.887521

25.890796

413.0

4.0

44.0

226.0

134.0

sample_639

29.0

NaN

102

NaN

73.0

103.0

104.0

105.0

74.0

1.0

...

1.290994

0.00000

18.009257

29.231642

25.514702

414.0

3.0

43.0

226.0

135.0

sample_483

38.0

NaN

100

NaN

62.0

102.0

103.0

104.0

64.0

2.0

...

1.707825

0.57735

9.165151

24.617293

22.479620

409.0

4.0

30.0

207.0

118.0

sample_617

29.0

NaN

101

NaN

72.0

103.0

104.0

105.0

74.0

2.0

...

1.707825

0.57735

17.785762

28.887521

25.890796

413.0

4.0

44.0

226.0

134.0

5 rows × 36 columns

==== 时间相关特征

时间特征提取和温度的思路基本一致，也是四个过程中分别计算时间差，只不过对于时间差我们需要做一个处理。最后也要提取统计特征

+*In[24]:*+
[source, ipython3]

```
# 时间计算方式转换, 这个是为了时间越界，比如 21:00 到后一天的 2:00， 这种后者减前者肯定是个负数，我们需要转一下
def duration_outer(series1, series2):
    duration = series2 - series1
    duration = np.where(duration<0, duration+24*60, duration)
    duration = np.where(duration>12*60, 24*60-duration, duration)
    duration = np.where(duration>6*60, 12*60-duration, duration)

    return duration
```

+*In[25]:*+
[source, ipython3]

```
raw = df_trn_tst.copy()
```

```
df = pd.DataFrame(raw['样本 id'])

# 加热过程
df['P1_S1_A5_0T'] = raw['A5_t'] # 初始时刻
df['P1_S2_A9_2T'] = raw['A9_t'] # 初始时刻
df['P1_T1_T0_D'] = duration_outer(raw['A7_t'], raw['A5_t']) # 初次测温时间差
df['P1_T2_T1_D'] = duration_outer(raw['A9_t'], raw['A7_t']) # 二次测温时间差
df['P1_T2_T0_K_D'] = duration_outer(raw['A9_t'], raw['A5_t']) # 开始加热至沸腾时间差

# 水解过程
df['P2_S1_A11_3T'] = raw['A11_t'] # 水解开始时刻
df['P2_S1_A16_5T'] = raw['A16_t'] # 水解结束时刻

df['P2_T3_T0_K_D'] = duration_outer(raw['A11_t'], raw['A5_t']) # 开始加热至投料时间差
df['P2_T3_T2_K_D'] = duration_outer(raw['A11_t'], raw['A9_t']) # 恒温至投料投料时间差
# df['P2_T4_T3_D'] = raw['A14_t'] - raw['A11_t'] # 水解初次测温时间差
# df['P2_T5_T4_D'] = raw['A16_t'] - raw['A14_t'] # 水解结束时间差
df['P2_T5_T3_K_D'] = duration_outer(raw['A16_t'], raw['A11_t']) # 水解时间差

# 脱色过程
df['P3_S1_A20_6T'] = raw['A20_at'] # 中和开始时刻
df['P3_S2_A25_7T'] = raw['A24_t'] # 保温时刻

df['P3_T6_T5_K_D'] = duration_outer(raw['A20_at'], raw['A16_t'])
# 水解结束至中和间歇时间
df['P3_T6_T6_K_D'] = duration_outer(raw['A20_bt'], raw['A20_at'])
# 酸碱度中和时间
df['P3_T7_T6_D'] = duration_outer(raw['A24_t'], raw['A20_bt'])
# 中和结束至脱色间歇时间
df['P3_T8_T7_K_D'] = duration_outer(raw['A26_t'], raw['A24_t'])
# 脱色保温时间
df['P3_T9_T8_D'] = duration_outer(raw['A28_at'], raw['A26_t'])
# 脱色至抽滤间歇时间
df['P3_T9_T9_K_D'] = duration_outer(raw['A28_bt'], raw['A28_at'])
# 抽滤时间
df['P3_T9_T5_1D'] = duration_outer(raw['A28_bt'], raw['A16_t'])
df['P3_T9_T6_2D'] = duration_outer(raw['A28_bt'], raw['A20_at'])
# 脱色总时间
```

```
# 结晶过程
df['P4_S1_B4_10T'] = raw['B4_at']  # 酸化开始时刻
df['P4_S2_B5_11T'] = raw['B5_t']  # 结晶开始时刻
df['P4_S3_B7_12T'] = raw['B7_t']  # 结晶结束时刻

df['P4_T10_T9_D'] = duration_outer(raw['B4_at'], raw['A28_bt'])
# 抽滤结束至酸化间歇时间
df['P4_T10_T10_K_D'] = duration_outer(raw['B4_bt'], raw['B4_at'])
# 酸化时间
df['P4_T11_T10_K_D'] = duration_outer(raw['B5_t'], raw['B4_bt'])
# 酸化至结晶间歇时间
df['P4_T12_T11_K_D'] = duration_outer(raw['B7_t'], raw['B5_t'])
# 自然结晶时间
df['P4_T12_T9_1D'] = duration_outer(raw['B7_t'], raw['A28_bt'])
df['P4_T12_T10_2D'] = duration_outer(raw['B7_t'], raw['B4_at'])
# 结晶总时间

# 甩滤过程
df['P5_S1_B9_13T'] = raw['B9_at']  # 甩滤开始时刻
df['P5_S3_B12_15T'] = np.where(
    raw['B11_bt'].isnull(),
    np.where(raw['B10_bt'].isnull(), raw['B9_bt'], raw['B10_bt']),
    raw['B11_bt'])  # 甩滤结束时刻
df['P5_T13_T12_D'] = duration_outer(raw['B9_at'], raw['B7_t'])
# 酸化结束至甩滤间歇时间
df['P5_T13_T13_K_D'] = duration_outer(raw['B9_bt'], raw['B9_at'])
# 基本甩滤时间
df['P5_T14_T13_D'] = duration_outer(raw['B10_at'], raw['B9_bt'])
# 基本甩滤至补充甩滤 1 间歇时间
df['P5_T14_T14_K_D'] = duration_outer(raw['B10_bt'], raw['B10_at'])
# 补充甩滤 1 时间
df['P5_T15_T14_D'] = duration_outer(raw['B11_at'], raw['B10_bt'])
# 补充甩滤 1 至补充甩滤 2 间歇时间
df['P5_T15_T13_K_D'] = duration_outer(raw['B11_bt'], raw['B11_at'])
# 补充甩滤 2 时间
df['P5_T15_T13_1D'] = \
    df[['P5_T13_T13_K_D',                                'P5_T14_T14_K_D',
'P5_T13_T13_K_D']].sum(axis=1)
df['P5_T15_T12_2D'] = duration_outer(
    df['P5_S3_B12_15T'], df['P4_S3_B7_12T'])
df['P5_T15_T12_3D'] = duration_outer(
    df['P5_S3_B12_15T'], df['P5_S1_B9_13T'])
# 总甩滤时间
```

```
# 总流程时长
df['P5_T15_T1_4D'] = \
    df[['P5_T15_T12_2D', 'P4_T12_T9_1D', 'P3_T9_T5_1D',
        'P2_T3_T0_K_D', 'P2_T5_T3_K_D']].sum(axis=1)
_funcs = ['mean', 'std', 'sum']
for _func in _funcs:
    df[f'P5__D_{_func}'] = \
        df[[_f for _f in df.columns if _f.endswith('_D')]].\
            abs().agg(_func, axis=1)
    df[f'P5_K_D_{_func}'] = \
        df[[_f for _f in df.columns if _f.endswith('_K_D')]]. \
            abs().agg(_func, axis=1)
    df[f'P5__D_{_func}'] = \
        df[[_f for _f in df.columns if _f.endswith('D')]]. \
            abs().agg(_func, axis=1)
df_duration = df.set_index('样本 id')
```

+*In[26]:*+
```
[source, ipython3]
----
df_duration.head()
----
```

+*Out[26]:*+
```
P1_S1_A5_0T

P1_S2_A9_2T

P1_T1_T0_D

P1_T2_T1_D

P1_T2_T0_K_D

P2_S1_A11_3T

P2_S1_A16_5T

P2_T3_T0_K_D
```

P2_T3_T2_K_D

P2_T5_T3_K_D

...

P5_T15_T13_1D

P5_T15_T12_2D

P5_T15_T12_3D

P5_T15_T1_4D

P5__D_mean

P5_K_D_mean

P5__D_std

P5_K_D_std

P5__D_sum

P5_K_D_sum

样本 id

sample_1528

810

930

NaN

NaN

120

990

1110

180

60

120

...

270.0

240.0

240.0

840.0

145.384615

90.000000

169.852425

63.639610

3780.0

1170.0

sample_1698

840

960

NaN

NaN

120

1020

1140

180

60

120

...

270.0

240.0

240.0

960.0

136.071429

90.000000

188.588113

76.258669

3810.0

1260.0

sample_639

840

960

NaN

NaN

120

1020

1140

180

60

120

...

270.0

240.0

240.0

900.0

123.214286

75.000000

173.654693

49.575118

3450.0

1050.0

sample_483

90

180

NaN

NaN

90

240

360

150

60

120

...

270.0

300.0

240.0

990.0

158.076923

73.846154

195.448596

46.822086

4110.0

960.0

sample_617

1320

0

NaN

NaN

120

60

180

180

60

120

...

270.0

240.0

240.0

900.0

123.214286

77.142857

173.846539

48.107024

3450.0

1080.0

5 rows × 47 columns

==== 水耗相关特征

和上面同样的处理思路，
但是这一块需要填充一些缺失，还要纠正一些不太正确的值

+*In[27]:*+
[source, ipython3]
---
na_value=405

df_trn_tst = df_trn_tst.copy()

```
df = pd.DataFrame(raw['样本 id'])
# 耗水
df['P2_W_1M'] = raw['A4']
df['P2_W_2M'] = raw['A19']
# 耗盐酸
df['P3_H_1M'] = raw['A21'].fillna(50)
df['P4_H_2M'] = raw['B1'].fillna(320)
# 氢氧化钠
df['P2_N_1M'] = raw['A3'].fillna(na_value)
# 4-氰基吡啶
df['P2_C_1M'] = raw['A1']

df['P5_W_3M'] = raw['B12'].fillna(1200)
df['P5_W_1M'] = df['P2_W_1M'] + df['P2_W_2M']
df['P5_W_3M'] = df['P2_W_1M'] + df['P2_W_2M'] + df['P5_W_3M']
df['P5_H_1M'] = df['P3_H_1M'] + df['P4_H_2M']
df['P5_M_OM'] = raw['A1'] + df['P2_N_1M'] + df['P5_W_1M'] + df['P4_H_2M']
df['P5_M_1M'] = df['P5_M_OM'] + df['P5_W_3M']
df['P5_M_2M'] = df['P5_M_1M'] + df['P3_H_1M']
# 理论产出
df['P5_O_1M'] = raw['B14']
df['P5_O_5M'] = raw['B14'].replace(418, 420).replace(405, 400).\
    replace(395, 390).replace(392, 390).replace(387, 380).\
    replace(385, 380).replace(370, 360).replace(350, 360).\
    replace(350, 360).replace(340, 360).replace(290, 280).\
    replace(260, 280).replace(256, 280)
_fs = [_f for _f in df.columns if _f.endswith('M')]
for _f in _fs[:-2]:
    df[f'{_f}_P5_O_1M_R'] = df['P5_O_1M'] / df[_f]
    df[f'{_f}_P5_O_5M_R'] = df['P5_O_5M'] / df[_f]
for i in range(len(_fs[:6])):
    _f, _sub_fs = _fs[i], _fs[(i+1):6]
    for _f_div in _sub_fs:
        df[f'{_f}_{_f_div}_R'] = df[_f] / df[_f_div]
df_materials = df.set_index('样本 id')
---


+*In[28]:*+
[source, ipython3]
---
df_materials.head()
---
```

+*Out[28]:*+

P2_W_1M

P2_W_2M

P3_H_1M

P4_H_2M

P2_N_1M

P2_C_1M

P5_W_3M

P5_W_1M

P5_H_1M

P5_M_0M

...

P2_W_2M_P3_H_1M_R

P2_W_2M_P4_H_2M_R

P2_W_2M_P2_N_1M_R

P2_W_2M_P2_C_1M_R

P3_H_1M_P4_H_2M_R

P3_H_1M_P2_N_1M_R

P3_H_1M_P2_C_1M_R

P4_H_2M_P2_N_1M_R

P4_H_2M_P2_C_1M_R

P2_N_1M_P2_C_1M_R

样本 id

sample_1528

700

300

50.0

350.0

405.0

300

1800.0

1000

400.0

2055.0

...

6.0

0.857143

0.740741

1.000000

0.142857

0.123457

0.166667

0.864198

1.166667

1.35

sample_1698

700

200

50.0

320.0

405.0

300

2100.0

900

370.0

1925.0

...

4.0

0.625000

0.493827

0.666667

0.156250

0.123457

0.166667

0.790123

1.066667

1.35

sample_639

700

200

50.0

320.0

405.0

300

2100.0

900

370.0

1925.0

...

4.0

0.625000

0.493827

0.666667

0.156250

0.123457

0.166667

0.790123

1.066667

1.35

sample_483

700

200

50.0

290.0

405.0

300

1700.0

900

340.0

1895.0

...

4.0

0.689655

0.493827

0.666667

0.172414

0.123457

0.166667

0.716049

0.966667

1.35

sample_617

700

200

50.0

320.0

405.0

300

2100.0

900

370.0

1925.0

...

4.0

0.625000

0.493827

0.666667

0.156250

0.123457

0.166667

0.790123

1.066667

1.35

5 rows × 53 columns

+*In[29]:*+
[source, ipython3]
---
raw = df_trn_tst.copy()
df = pd.DataFrame(raw['样本 id'])
df['P5_NOT_NUM_N'] = raw.iloc[:, 1:-1].notnull().sum(axis=1)
df['P5_PH_1N'] = raw['A22']
df['P5_PH_2N'] = raw['A23']
df['P5_PH_2N'] = raw['B2']
df['P5_A7_1N'] = raw['A7_t'].isnull().astype(int)
df['P5_O_2M'] = (raw['B14'] <= 360).astype(int)
df['P5_1_3M'] = raw['B13']
df_interact = df.set_index('样本 id')

+*In[30]:*+
[source, ipython3]
---
df_interact.head()

+*Out[30]:*+

P5_NOT_NUM_N

P5_PH_1N

P5_PH_2N

P5_A7_1N

P5_O_2M

P5_1_3M

样本 id

sample_1528

42

9.0

3.5

1

0

0.15

sample_1698

44

9.0

3.5

1

0

0.15

sample_639

44

9.0

3.5

1

0

0.15

sample_483

42

10.0

3.5

1

0

0.15

sample_617

44

9.0

3.5

1

0

0.15

=== 合并所有特征

由于上面提取特征时都把索引重置成一样的了，这里合并的时候，直接相连，然后再重置索引即可。

+*In[31]:*+
[source, ipython3]
---
df_feature = pd.concat([df_materials, df_duration, df_temperature, df_interact], axis=1).reset_index()

+*In[32]:*+

[source, ipython3]

这样总共是做了 143 个特征出来
df_feature.head()

+*Out[32]:*+

[cols=",,,,,,,,,,,,,,,,,,,,,,",options="header",]
|===
==
| |样本 id |P2_W_1M |P2_W_2M |P3_H_1M |P4_H_2M |P2_N_1M |P2_C_1M |P5_W_3M
|P5_W_1M |P5_H_1M |... |P2_D3-D5_sum |P2_C1-C12_KD_ABS_sum
|P2_C1-C12_D_sum |P2_LARGE_KD_sum |P5_NOT_NUM_N |P5_PH_1N |P5_PH_2N
|P5_A7_1N |P5_O_2M |P5_1_3M
|0 |sample_1528 |700 |300 |50.0 |350.0 |405.0 |300 |1800.0 |1000 |400.0
|... |4.0 |27.0 |191.0 |113.0 |42 |9.0 |3.5 |1 |0 |0.15

|1 |sample_1698 |700 |200 |50.0 |320.0 |405.0 |300 |2100.0 |900 |370.0
|... |4.0 |44.0 |226.0 |134.0 |44 |9.0 |3.5 |1 |0 |0.15

|2 |sample_639 |700 |200 |50.0 |320.0 |405.0 |300 |2100.0 |900 |370.0
|... |3.0 |43.0 |226.0 |135.0 |44 |9.0 |3.5 |1 |0 |0.15

|3 |sample_483 |700 |200 |50.0 |290.0 |405.0 |300 |1700.0 |900 |340.0
|... |4.0 |30.0 |207.0 |118.0 |42 |10.0 |3.5 |1 |0 |0.15

|4 |sample_617 |700 |200 |50.0 |320.0 |405.0 |300 |2100.0 |900 |370.0
|... |4.0 |44.0 |226.0 |134.0 |44 |9.0 |3.5 |1 |0 |0.15
|===
==

5 rows × 143 columns

+*In[33]:*+
[source, ipython3]

"""把训练集和测试集给他分开"""
df_trn = df_feature.iloc[:len(df_trn)].reset_index(drop=True)
df_trn['收率'] = df_target
df_tst = df_feature.iloc[len(df_trn):].reset_index(drop=True)

```
df_tst['收率'] = np.nan
```

+*In[34]:*+
[source, ipython3]
```
df_trn.head()
```

+*Out[34]:*+
```
[cols=",,,,,,,,,,,,,,,,,,,,,,",options="header",]
|=======================================================================
==
| |样本 id |P2_W_1M |P2_W_2M |P3_H_1M |P4_H_2M |P2_N_1M |P2_C_1M |P5_W_3M
|P5_W_1M |P5_H_1M |... |P2_C1-C12_KD_ABS_sum |P2_C1-C12_D_sum
|P2_LARGE_KD_sum |P5_NOT_NUM_N |P5_PH_1N |P5_PH_2N |P5_A7_1N |P5_O_2M
|P5_1_3M |收率
|0 |sample_1528 |700 |300 |50.0 |350.0 |405.0 |300 |1800.0 |1000 |400.0
|... |27.0 |191.0 |113.0 |42 |9.0 |3.5 |1 |0 |0.15 |0.879

|1 |sample_1698 |700 |200 |50.0 |320.0 |405.0 |300 |2100.0 |900 |370.0
|... |44.0 |226.0 |134.0 |44 |9.0 |3.5 |1 |0 |0.15 |0.902

|2 |sample_639 |700 |200 |50.0 |320.0 |405.0 |300 |2100.0 |900 |370.0
|... |43.0 |226.0 |135.0 |44 |9.0 |3.5 |1 |0 |0.15 |0.936

|3 |sample_483 |700 |200 |50.0 |290.0 |405.0 |300 |1700.0 |900 |340.0
|... |30.0 |207.0 |118.0 |42 |10.0 |3.5 |1 |0 |0.15 |0.902

|4 |sample_617 |700 |200 |50.0 |320.0 |405.0 |300 |2100.0 |900 |370.0
|... |44.0 |226.0 |134.0 |44 |9.0 |3.5 |1 |0 |0.15 |0.983
|=======================================================================
==
```

5 rows × 144 columns

+*In[35]:*+
[source, ipython3]
```
```

"""id 中提取后面的数出来"""
for _df in [df_trn, df_tst]:
    _df.insert(1,   'id',   _df['   样   本
id'].str.split('_').str[1].astype(float))

+*In[36]:*+
[source, ipython3]
----
df_trn.head()

+*Out[36]:*+

[cols=",,,,,,,,,,,,,,,,,,,,,,",options="header",]
|===
==
| |样本 id |id |P2_W_1M |P2_W_2M |P3_H_1M |P4_H_2M |P2_N_1M |P2_C_1M
|P5_W_3M |P5_W_1M |... |P2_C1-C12_KD_ABS_sum |P2_C1-C12_D_sum
|P2_LARGE_KD_sum |P5_NOT_NUM_N |P5_PH_1N |P5_PH_2N |P5_A7_1N |P5_O_2M
|P5_1_3M |收率
|0 |sample_1528 |1528.0 |700 |300 |50.0 |350.0 |405.0 |300 |1800.0 |1000
|... |27.0 |191.0 |113.0 |42 |9.0 |3.5 |1 |0 |0.15 |0.879

|1 |sample_1698 |1698.0 |700 |200 |50.0 |320.0 |405.0 |300 |2100.0 |900
|... |44.0 |226.0 |134.0 |44 |9.0 |3.5 |1 |0 |0.15 |0.902

|2 |sample_639 |639.0 |700 |200 |50.0 |320.0 |405.0 |300 |2100.0 |900
|... |43.0 |226.0 |135.0 |44 |9.0 |3.5 |1 |0 |0.15 |0.936

|3 |sample_483 |483.0 |700 |200 |50.0 |290.0 |405.0 |300 |1700.0 |900
|... |30.0 |207.0 |118.0 |42 |10.0 |3.5 |1 |0 |0.15 |0.902

|4 |sample_617 |617.0 |700 |200 |50.0 |320.0 |405.0 |300 |2100.0 |900
|... |44.0 |226.0 |134.0 |44 |9.0 |3.5 |1 |0 |0.15 |0.983
|===
==

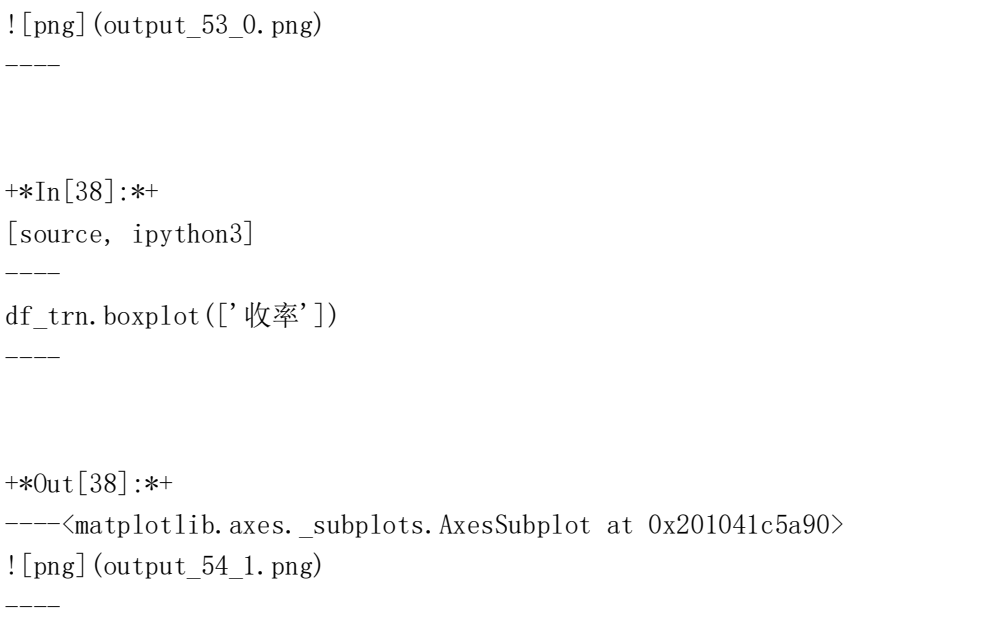
5 rows × 145 columns

=== 筛选常规数据

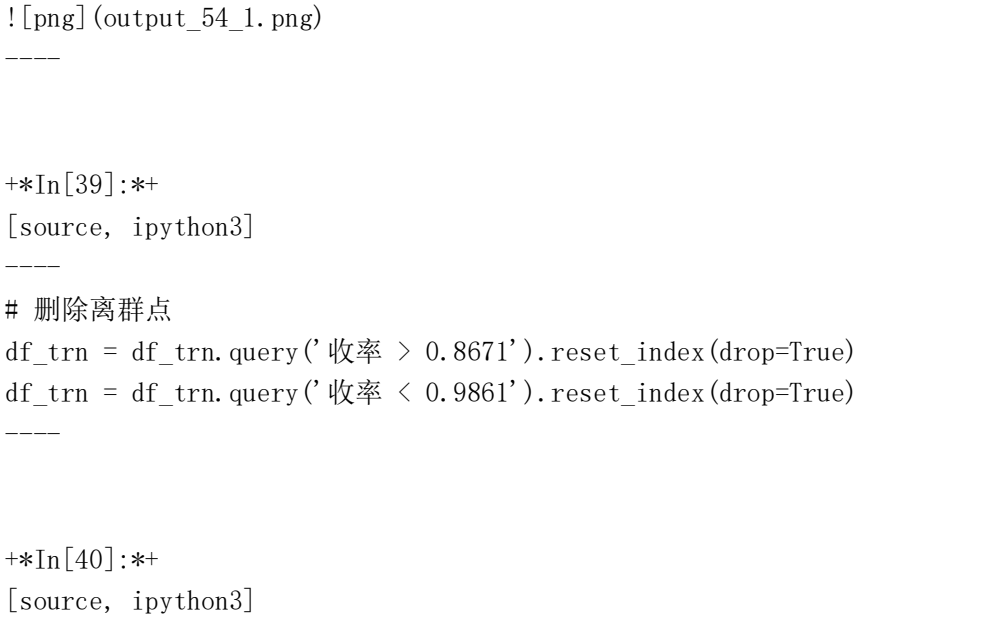
+*In[37]:*+
[source, ipython3]
----
import matplotlib.pyplot as plt

df_trn['收率'].plot(kind='hist')
plt.xlim(0.8, 1)
plt.show()

+*Out[37]:*+



+*In[38]:*+
[source, ipython3]
----
df_trn.boxplot(['收率'])

+*Out[38]:*+
----<matplotlib.axes._subplots.AxesSubplot at 0x201041c5a90>


+*In[39]:*+
[source, ipython3]

删除离群点
df_trn = df_trn.query('收率 > 0.8671').reset_index(drop=True)
df_trn = df_trn.query('收率 < 0.9861').reset_index(drop=True)

+*In[40]:*+
[source, ipython3]

```
df_trn.boxplot(['收率'])
```

+*Out[40]:*+
```
<matplotlib.axes._subplots.AxesSubplot at 0x201041fd080>
```


+*In[41]:*+
```
[source, ipython3]
df_trn.head()
```

+*Out[41]:*+
```
[cols=",,,,,,,,,,,,,,,,,,,,,,",options="header",]
|=======================================================================
|  |样本 id |id |P2_W_1M |P2_W_2M |P3_H_1M |P4_H_2M |P2_N_1M |P2_C_1M
|P5_W_3M |P5_W_1M |... |P2_C1-C12_KD_ABS_sum |P2_C1-C12_D_sum
|P2_LARGE_KD_sum |P5_NOT_NUM_N |P5_PH_1N |P5_PH_2N |P5_A7_1N |P5_O_2M
|P5_1_3M |收率
|0 |sample_1528 |1528.0 |700 |300 |50.0 |350.0 |405.0 |300 |1800.0 |1000
|... |27.0 |191.0 |113.0 |42 |9.0 |3.5 |1 |0 |0.15 |0.879

|1 |sample_1698 |1698.0 |700 |200 |50.0 |320.0 |405.0 |300 |2100.0 |900
|... |44.0 |226.0 |134.0 |44 |9.0 |3.5 |1 |0 |0.15 |0.902

|2 |sample_639 |639.0 |700 |200 |50.0 |320.0 |405.0 |300 |2100.0 |900
|... |43.0 |226.0 |135.0 |44 |9.0 |3.5 |1 |0 |0.15 |0.936

|3 |sample_483 |483.0 |700 |200 |50.0 |290.0 |405.0 |300 |1700.0 |900
|... |30.0 |207.0 |118.0 |42 |10.0 |3.5 |1 |0 |0.15 |0.902

|4 |sample_617 |617.0 |700 |200 |50.0 |320.0 |405.0 |300 |2100.0 |900
|... |44.0 |226.0 |134.0 |44 |9.0 |3.5 |1 |0 |0.15 |0.983
|=======================================================================

5 rows × 145 columns
```

== 模型的建立和训练

本次模型使用 xgboost，这个是数据比赛的神器了

+*In[51]:*+
[source, ipython3]

```
def xgb_cv(train, test, params, fit_params, feature_names, nfold, seed):
    # 创建结果 df
    train_pred = pd.DataFrame({
        'id': train['样本 id'],
        'true': train['收率'],
        'pred': np.zeros(len(train))
    })

    # 测试提交结果
    test_pred = pd.DataFrame({'id':test['样本 id'],
'pred':np.zeros(len(test))})
    # 交叉验证
    kfolder = KFold(n_splits=nfold, shuffle=True, random_state=seed)
    # 构造测试 DMatrix
    xgb_tst = xgb.DMatrix(data=test[feature_names])
    print('\n')

    # 遍历 cv 的每一折数据，通过索引来指定
    for fold_id, (trn_idx, val_idx) in enumerate(kfolder.split(train['
收率'])):
        # 构造当前训练的 DMatrix
        xgb_trn = xgb.DMatrix(
            train.iloc[trn_idx][feature_names],
            train.iloc[trn_idx]['收率']
        )
        # 构造当前验证的 DMatrix
        xgb_val = xgb.DMatrix(
            train.iloc[val_idx][feature_names],
            train.iloc[val_idx]['收率']
        )
        # 训练回归模型
        xgb_reg = xgb.train(params=params, dtrain=xgb_trn, **fit_params,
                            evals=[(xgb_trn, 'train'), (xgb_val,
'valid')])
```

```
        # 得到验证结果
        val_pred = xgb_reg.predict(
            xgb.DMatrix(train.iloc[val_idx][feature_names]),
            ntree_limit = xgb_reg.best_ntree_limit
        )
        train_pred.loc[val_idx, 'pred'] = val_pred
        # 这里是 k 折求了个平均
        test_pred['pred']          +=          xgb_reg.predict(xgb_tst,
ntree_limit=xgb_reg.best_ntree_limit) / nfold
    print('\nCV LOSS: ', mse(train_pred['true'], train_pred['pred']),
'\n')
    return test_pred
---
```

设置训练参数

```
+*In[49]:*+
[source, ipython3]
---
fit_params = {'num_boost_round':10800,
              'verbose_eval': 300,
              'early_stopping_rounds': 360
             }
params_xgb = {'eta':0.01, 'max_depth': 7, 'subsample':0.8,
              'booster':'gbtree', 'colsample_bytree':0.8,
              'objective':'reg:linear', 'silent':True, 'nthread':4
             }
---


+*In[52]:*+
[source, ipython3]
---
# 开始训练
pred_xgb_a   =   xgb_cv(df_trn,   df_tst,   params_xgb,   fit_params,
df_trn.columns.tolist()[1:-1], 5, 0)
---


+*Out[52]:*+
---
```

```
[0] train-rmse:0.42052  valid-rmse:0.417952
Multiple eval metrics have been passed: 'valid-rmse' will be used for
early stopping.

Will train until valid-rmse hasn't improved in 360 rounds.
[300]   train-rmse:0.023733 valid-rmse:0.023684
[600]   train-rmse:0.006536 valid-rmse:0.011536
[900]   train-rmse:0.004731 valid-rmse:0.011757
Stopping. Best iteration:
[569]   train-rmse:0.006873 valid-rmse:0.011534

[0] train-rmse:0.419812 valid-rmse:0.420785
Multiple eval metrics have been passed: 'valid-rmse' will be used for
early stopping.

Will train until valid-rmse hasn't improved in 360 rounds.
[300]   train-rmse:0.023766 valid-rmse:0.025733
[600]   train-rmse:0.006571 valid-rmse:0.012127
[900]   train-rmse:0.004669 valid-rmse:0.012029
Stopping. Best iteration:
[771]   train-rmse:0.005343 valid-rmse:0.012019

[0] train-rmse:0.419963 valid-rmse:0.420191
Multiple eval metrics have been passed: 'valid-rmse' will be used for
early stopping.

Will train until valid-rmse hasn't improved in 360 rounds.
[300]   train-rmse:0.023614 valid-rmse:0.025042
[600]   train-rmse:0.006238 valid-rmse:0.01244
[900]   train-rmse:0.004508 valid-rmse:0.012215
[1200]  train-rmse:0.003474 valid-rmse:0.012207
Stopping. Best iteration:
[1060]  train-rmse:0.003912 valid-rmse:0.012189

[0] train-rmse:0.420254 valid-rmse:0.419025
Multiple eval metrics have been passed: 'valid-rmse' will be used for
early stopping.

Will train until valid-rmse hasn't improved in 360 rounds.
[300]   train-rmse:0.023809 valid-rmse:0.024823
[600]   train-rmse:0.006452 valid-rmse:0.010959
[900]   train-rmse:0.004604 valid-rmse:0.010781
[1200]  train-rmse:0.003514 valid-rmse:0.010848
Stopping. Best iteration:
```

```
[873]   train-rmse:0.004744 valid-rmse:0.010775

[0] train-rmse:0.419487 valid-rmse:0.422069
Multiple eval metrics have been passed: 'valid-rmse' will be used for
early stopping.

Will train until valid-rmse hasn't improved in 360 rounds.
[300]   train-rmse:0.023863 valid-rmse:0.024234
[600]   train-rmse:0.006793 valid-rmse:0.010373
[900]   train-rmse:0.00486  valid-rmse:0.010267
[1200]  train-rmse:0.003688 valid-rmse:0.010304
Stopping. Best iteration:
[843]   train-rmse:0.00512  valid-rmse:0.010259


CV LOSS:  0.00012948923245158987

----

== 模型的测试与结果


+*In[54]:*+
[source, ipython3]
----
# 得到预测结果
df_tst_a['收率'] = pred_xgb_a['pred'].values
----


+*In[55]:*+
[source, ipython3]
----
df_tst_a.head()
----


+*Out[55]:*+
----
[cols=",,,,,,,,,,,,,,,,,,,,,,",options="header",]
|=======================================================================
==
| |样本 id |A1 |A2 |A3 |A4 |A5 |A6 |A7 |A8 |A9 |... |B6 |B7 |B8 |B9 |B10
|B11 |B12 |B13 |B14 |收率
```

|0 |sample_1656 |300 |NaN |405.0 |700 |6:00:00 |29 |NaN |NaN |8:00:00 |... |79 |17:00:00 |45 |17:00-18:30 |18:30-20:00 |20:00-21:00 |1200 |0.15 |400 |0.906113

|1 |sample_1548 |300 |NaN |405.0 |700 |12:30:00 |39 |12:50:00 |80.0 |14:20:00 |... |65 |10:00:00 |45 |12:00-13:00 |14:00-15:30 |NaN |800 |0.15 |385 |0.879397

|2 |sample_769 |300 |NaN |405.0 |700 |6:00:00 |80 |NaN |NaN |8:00:00 |... |80 |17:00:00 |45 |17:00-20:00 |NaN |NaN |1200 |0.15 |440 |0.934677

|3 |sample_1881 |300 |NaN |405.0 |700 |22:00:00 |29 |NaN |NaN |0:00:00 |... |80 |9:00:00 |45 |9:00-10:30 |10:30-12:00 |12:00-13:00 |1200 |0.15 |400 |0.903162

|4 |sample_1807 |300 |NaN |405.0 |700 |22:00:00 |30 |NaN |NaN |0:00:00 |... |79 |9:00:00 |45 |9:00-10:30 |10:30-12:00 |12:00-13:00 |1200 |0.15 |400 |0.928358
|==========
==

5 rows × 44 columns

== 总结

* Pandas 的数值访问，即.loc, .iloc, .ix 访问数据的区别
* np.where
* pandas.query
* pandas.set_index 和 reset_index
* xgboost 的使用
* 一些 tricks

=== Pandas 的数值访问，.loc, .iloc, .ix 访问数据的区别

+*In[56]:*+
[source, ipython3]
----
import pandas as pd
data =
pd.DataFrame({'A':[1,2,3],'B':[4,5,6],'C':[7,8,9]},index=["a","b","c"])
data

+*Out[56]:*+

[cols=",,,",options="header",]
|==========
| |A |B |C
|a |1 |4 |7
|b |2 |5 |8
|c |3 |6 |9
|==========

+*In[58]:*+
[source, ipython3]

"""
.loc[],中括号里面是先行后列，以逗号分割，行和列分别是行标签和列标签
"""
data.loc["b", "B"]
data.loc["b":"c", "B":"C"]   # 这种是包括后面的那个标签的

+*Out[58]:*+

[cols=",,",options="header",]
|=======
| |B |C
|b |5 |8
|c |6 |9
|=======

.loc 是用行列标签来进行选择数据的。那么，我们会想，那我们只知道要第几行，第几列的数据呢，这该怎么办，刚好，.iloc 就是干这个事的

+*In[59]:*+
[source, ipython3]

"""
.iloc[]与 loc 一样，中括号里面也是先行后列，行列标签用逗号分割，
与 loc 不同的之处是，.iloc 是根据行数与列数来索引的"""
data.iloc[1:3, 1:3]   # 此处区间是前闭后开了， 而 loc 是前闭后闭

```
----
```

```
+*Out[59]:*+
----
[cols=",,",options="header",]
|=======
| |B |C
|b |5 |8
|c |6 |9
|=======
----
```

```
+*In[62]:*+
[source, ipython3]
----
""".ix 我发现，上面两种用法他都可以，它既可以根据行列标签又可以根据行列数"""
data.ix[1,1]
data.ix["b":"c", "B":"C"]
----
```

```
+*Out[62]:*+
----
[cols=",,",options="header",]
|=======
| |B |C
|b |5 |8
|c |6 |9
|=======
----
```

```
=== numpy 的 where

* np.where(condition, x, y): 满足 condition， 输出 x， 不满足，输出 y
* np.where(condition), 只有条件 (condition)，没有 x 和 y，则输出满足条件 (即非 0) 元素的坐标
,这里的坐标以 tuple 的形式给出，通常原数组有多少维，输出的 tuple 中就包含几个数组，分别对应符合条件元素的各维坐标。
```

```
+*In[63]:*+
```

[source, ipython3]
----
aa = np.arange(10)
aa

+*Out[63]:*+
----array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])----

+*In[64]:*+
[source, ipython3]
----
np.where(aa, 1, -1)   # 大于 0，返回 1， 否则，返回-1

+*Out[64]:*+
----array([-1, 1, 1, 1, 1, 1, 1, 1, 1, 1])----

+*In[66]:*+
[source, ipython3]
----
a = np.array([2, 4, 6, 8, 10])
np.where(a>5)   # 返回索引
a[np.where(a>5)]   # 选择符合条件的元素

+*Out[66]:*+
----array([6, 8, 10])----

=== pandas.query

这个是字符串表达式查询，基于 DataFrame 列的计算代数式。对于过滤的操作，可以使用 query()方法。注意支持 string，不支持布尔类型，
与 query 类似的一个函数是 eval，
是使用字符串表达式来计算对 DataFrame 的操作，支持算术，比较，位，对象索引，列间计算等，
这俩都是高性能函数，后者遇到的时候再整理吧。

+*In[67]:*+
[source, ipython3]
----
import pandas as pd
d={
    'name':['xiao','dan','qi'],
    'sex':['male','female','male'],
    'age':[23,24,24]
}
df=pd.DataFrame(d)
df

+*Out[67]:*+

[cols=",,,",options="header",]
|==================
| |name |sex |age
|0 |xiao |male |23
|1 |dan |female |24
|2 |qi |male |24
|==================

+*In[68]:*+
[source, ipython3]
----
df query('age'>23) # 这个会报错
df.query('age>23')

+*Out[68]:*+

[cols=",,,",options="header",]
|==================
| |name |sex |age
|1 |dan |female |24
|2 |qi |male |24
|==================

对于筛选，numpy 有 where， pandas 有 query。

=== pandas.set_index 和 reset_index

* DataFrame 可以通过 set_index 方法，可以设置单索引和复合索引。
>DataFrame.set_index(keys, drop=True, append=False, inplace=False, verify_integrity=False) > *
keys：列标签或列标签/数组列表，需要设置为索引的列 > *
drop：默认为 True，删除用作新索引的列 > *
append：默认为 False，是否将列附加到现有索引 > *
inplace：默认为 False，适当修改 DataFrame(不要创建新对象) > *
verify_integrity：默认为 false，检查新索引的副本。否则，请将检查推迟到必要时进行。将其设置为 false 将提高该方法的性能。
* reset_index 可以还原索引，从新变为默认的整型索引
>DataFrame.reset_index(level=None, drop=False, inplace=False, col_level=0, col_fill=”) > *
level：int、str、tuple 或 list，默认无，仅从索引中删除给定级别。默认情况下移除所有级别。控制了具体要还原的那个等级的索引
> * drop：drop 为 False 则索引列会被还原为普通列，否则会丢失 > *
inplace：默认为 false，适当修改 DataFrame(不要创建新对象) > *
col_level：int 或 str，默认值为 0，如果列有多个级别，则确定将标签插入到哪个级别。默认情况下，它将插入到第一级。
> *
col_fill：对象，默认 ‘’ ，如果列有多个级别，则确定其他级别的命名方式。如果没有，则重复索引名

+*In[69]:*+
[source, ipython3]

```
import pandas as pd
df = pd.DataFrame({ 'A': ['A0', 'A1', 'A2', 'A3'],
                    'B': ['B0', 'B1', 'B2', 'B3'],
                    'C': ['C0', 'C1', 'C2', 'C3'],
                    'D': ['D0', 'D1', 'D2', 'D3']})
df
```

+*Out[69]:*+

[cols=",,,,",options="header",]
|=================
| |A |B |C |D

|0 |A0 |B0 |C0 |D0
|1 |A1 |B1 |C1 |D1
|2 |A2 |B2 |C2 |D2
|3 |A3 |B3 |C3 |D3
|=================

+*In[73]:*+
[source, ipython3]
----
drop_t = df.set_index('A',drop=True, append=False, inplace=False, verify_integrity=False)
drop_t

+*Out[73]:*+

B

C

D

A

A0

B0

C0

D0

A1

B1

C1

D1

A2

B2

C2

D2

A3

B3

C3

D3

+*In[75]:*+
[source, ipython3]

这样就可以通过索引列名来访问元素了
drop_t.loc['A0':'A2']

+*Out[75]:*+

B

C

D

A

A0

B0

C0

D0

A1

B1

C1

D1

A2

B2

C2

D2

+*In[76]:*+
[source, ipython3]
----
drop=False， 不会删除原列
no_drop_t = df.set_index('A',drop=False, append=False, inplace=False, verify_integrity=False)
no_drop_t

+*Out[76]:*+

A

B

C

D

A

A0

A0

B0

C0

D0

A1

A1

B1

C1

D1

A2

A2

B2

C2

D2

A3

A3

B3

C3

D3

+*In[77]:*+
[source, ipython3]
----
reset_drop_t = drop_t.reset_index(drop=False) #索引列会被还原为普通列
reset_drop_t

+*Out[77]:*+

[cols=",,,,",options="header",]
|================
| |A |B |C |D
|0 |A0 |B0 |C0 |D0
|1 |A1 |B1 |C1 |D1
|2 |A2 |B2 |C2 |D2
|3 |A3 |B3 |C3 |D3
|================

+*In[78]:*+
[source, ipython3]
----
reset_no_drop_t = no_drop_t.reset_index(drop=True) #索引列会被还原为普通列
reset_no_drop_t

+*Out[78]:*+

[cols=",,,,",options="header",]
|================
| |A |B |C |D
|0 |A0 |B0 |C0 |D0
|1 |A1 |B1 |C1 |D1
|2 |A2 |B2 |C2 |D2
|3 |A3 |B3 |C3 |D3
|================

=== xgboost 的使用

xgboost 的核心： 一开始是只构造一棵树，然后不断的往里面加树，
在加的时候要使得没加一次，效果都会比原来更好。
xgboost 的优化目标两个部分：
一个是与真实值的一个差距，其次还有一个正则项（不过这个正则项既包含 w，也包含叶子节点的个数）。
我们如何选择每一轮加入什么样的模型呢？ 答案也是非常直接，
选取一个 f 来使得我们的目标函数尽量最大的降低。会有个残差计算的。

这一块我后面会出个博客完成推导部分，可以先参考这篇博客:https://www.cnblogs.com/LHWorldBlog/p/9181121.html[【机器学习】-xgboost从初识到应用]。这里只介绍使用：

==== 基本的使用

安装：默认可以通过pip安装，若是安装不上可以通过https://www.lfd.uci.edu/~gohlke/pythonlibs/网站下载相关安装包,将安装包拷贝到Anacoda3的安装目录的Scrripts目录下，
然后pip install 安装包安装。 我们下面用xgboost做一个简单的分类任务。

```
+*In[50]:*+
[source, ipython3]
----
import pandas as pd

columns = ["f"+str(i) for i in range(8)]
columns.append('label')
data = pd.read_csv('./pima-indians-diabetes.csv',names=columns)
data.head()
----
```

```
+*Out[50]:*+
----
[cols=",,,,,,,,,",options="header",]
|=========================================
| |f0 |f1 |f2 |f3 |f4 |f5 |f6 |f7 |label
|0 |6 |148 |72 |35 |0 |33.6 |0.627 |50 |1
|1 |1 |85 |66 |29 |0 |26.6 |0.351 |31 |0
|2 |8 |183 |64 |0 |0 |23.3 |0.672 |32 |1
|3 |1 |89 |66 |23 |94 |28.1 |0.167 |21 |0
|4 |0 |137 |40 |35 |168 |43.1 |2.288 |33 |1
|=========================================
----
```

```
+*In[43]:*+
[source, ipython3]
----
"""导入包"""
from numpy import loadtxt
from xgboost import XGBClassifier
```

```
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

"""导入数据"""
dataset = loadtxt('./pima-indians-diabetes.csv', delimiter=",")
# 划分数据集
X = dataset[:, 0:8]
Y = dataset[:, 8]

seed = 7
test_size = 0.33
x_train, x_test, y_train, y_test = train_test_split(X, Y,
test_size=test_size, random_state=seed)

model = XGBClassifier()
model.fit(x_train, y_train)

y_pred = model.predict(x_test)

predictions = [round(value) for value in y_pred]

accuracy = accuracy_score(y_test, predictions)
accuracy
----
```

```
+*Out[43]:*+
----0.7795275590551181----
```

上面就是一个简单的 xgboost 的初步使用方式，我们还可以每次插入一棵树，看看效果，这个要设置 eval_set,
训练的时候要给它指定上

```
+*In[44]:*+
[source, ipython3]
----
eval_set = [(x_test, y_test)]
model.fit(x_train, y_train, early_stopping_rounds=10,
eval_metric="logloss", eval_set=eval_set, verbose=True)

y_pred = model.predict(x_test)

predictions = [round(value) for value in y_pred]
```

```
accuracy = accuracy_score(y_test, predictions)
accuracy
```

+*Out[44]:*+

```
[0] validation_0-logloss:0.660186
Will train until validation_0-logloss hasn't improved in 10 rounds.
[1] validation_0-logloss:0.634854
[2] validation_0-logloss:0.61224
[3] validation_0-logloss:0.593118
[4] validation_0-logloss:0.578303
[5] validation_0-logloss:0.564942
[6] validation_0-logloss:0.555113
[7] validation_0-logloss:0.54499
[8] validation_0-logloss:0.539151
[9] validation_0-logloss:0.531819
[10]    validation_0-logloss:0.526065
[11]    validation_0-logloss:0.519769
[12]    validation_0-logloss:0.514979
[13]    validation_0-logloss:0.50927
[14]    validation_0-logloss:0.506086
[15]    validation_0-logloss:0.503565
[16]    validation_0-logloss:0.503591
[17]    validation_0-logloss:0.500805
[18]    validation_0-logloss:0.497605
[19]    validation_0-logloss:0.495328
[20]    validation_0-logloss:0.494777
[21]    validation_0-logloss:0.494274
[22]    validation_0-logloss:0.493333
[23]    validation_0-logloss:0.492211
[24]    validation_0-logloss:0.491936
[25]    validation_0-logloss:0.490578
[26]    validation_0-logloss:0.490895
[27]    validation_0-logloss:0.490646
[28]    validation_0-logloss:0.491911
[29]    validation_0-logloss:0.491407
[30]    validation_0-logloss:0.488828
[31]    validation_0-logloss:0.487867
[32]    validation_0-logloss:0.487297
[33]    validation_0-logloss:0.487562
[34]    validation_0-logloss:0.487789
```

```
[35]    validation_0-logloss:0.487962
[36]    validation_0-logloss:0.488218
[37]    validation_0-logloss:0.489582
[38]    validation_0-logloss:0.489334
[39]    validation_0-logloss:0.490968
[40]    validation_0-logloss:0.48978
[41]    validation_0-logloss:0.490704
[42]    validation_0-logloss:0.492369
Stopping. Best iteration:
[32]    validation_0-logloss:0.487297

0.7755905511811023----
```

上面就会显示每一次插入一棵树之后得到的结果，
我们还可以通过 xgboost 的 plot_importance 来观察特征的重要程度

```
+*In[45]:*+
[source, ipython3]
----
from xgboost import plot_importance
import matplotlib.pyplot as plt

plot_importance(model)
plt.show()
----
```

```
+*Out[45]:*+
----
![png](output_95_0.png)
----
```

下面介绍一下 xgboost 的一些参数： * `booster':`gbtree', 这个指定基分类器
*
`objective': `multi:softmax', 多分类的问题，
这个是优化目标，必须得有，因为 xgboost 里面有求一阶导数和二阶导数，其实就是这个。
* `num_class':10, 类别数，与 multisoftmax 并用 *
`gamma':损失下降多少才进行分裂， 控制叶子节点的个数 * `max_depth':12,
构建树的深度，越大越容易过拟合 * `lambda':2,
控制模型复杂度的权重值的 L2 正则化项参数，参数越大，模型越不容易过拟合。
*
`subsample':0.7, 随机采样训练样本 * `colsample_bytree':0.7,

生成树时进行的列采样 * `min_child_weight':3,
孩子节点中最小的样本权重和。如果一个叶子节点的样本权重和小于min_child_weight 则拆分过程结束
* `silent':0 ,设置成 1 则没有运行信息输出,最好是设置为 0. * `eta': 0.007,
如同学习率 * `seed':1000, * `nthread':7, cpu 线程数 xgb1 =
XGBClassifier(learning_rate =0.1, n_estimators=1000, max_depth=5,
min_child_weight=1, gamma=0, subsample=0.8, colsample_bytree=0.8,
objective= `binary:logistic', nthread=4, scale_pos_weight=1, seed=27)

下面就来看一个调参的技巧，使用 sklearn 的 GridSearchCV

+*In[52]:*+
[source, ipython3]

```
from sklearn.model_selection import GridSearchCV
from sklearn.model_selection import StratifiedKFold

model = XGBClassifier()
learning_rate = [0.0001, 0.001, 0.1, 0.2, 0.3]
param_grid = dict(learning_rate=learning_rate)
kfold = StratifiedKFold(n_splits=10, shuffle=True, random_state=7)
grid_search = GridSearchCV(model, param_grid, scoring="neg_log_loss",
n_jobs=-1, cv=kfold)
grid_result = grid_search.fit(x_train, y_train)

print("best: %f using %s" %(grid_result.best_score_,
grid_result.best_params_))

means = grid_result.cv_results_['mean_test_score']
params = grid_result.cv_results_['params']

for mean, param in zip(means, params):
    print("%f  with:  %r" % (mean, param))
```

+*Out[52]:*+

```
best: -0.506067 using {'learning_rate': 0.1}
-0.689932  with:  {'learning_rate': 0.0001}
-0.663696  with:  {'learning_rate': 0.001}
-0.506067  with:  {'learning_rate': 0.1}
-0.556035  with:  {'learning_rate': 0.2}
```

-0.621871 with： {'learning_rate': 0.3}

==== Xgboost Demo（七月在线）

七月在线课程: 高级工具 xgboost 建模实战 *
https://xgboost.readthedocs.io/en/latest/parameter.html[Xgboost 参数说明页面]
*
https://xgboost.readthedocs.io/en/latest/tutorials/param_tuning.html[Xgboost 调参官方指南]

===== 基本的用法

我们也是使用 xgboost 做一个分类任务，在这里还会接触一些新的东西。

+*In[54]:*+
[source, ipython3]
----
0 1:1 9:1 19:1 21:1 24:1 34:1 36:1 39:1 42:1 53:1 56:1 65:1 69:1 77:1 86:1 88:1 92:1 95:1 102:1 106:1 117:1 122:1
1 3:1 9:1 19:1 21:1 30:1 34:1 36:1 40:1 41:1 53:1 58:1 65:1 69:1 77:1 86:1 88:1 92:1 95:1 102:1 106:1 118:1 124:1
0 1:1 9:1 20:1 21:1 24:1 34:1 36:1 39:1 41:1 53:1 56:1 65:1 69:1 77:1 86:1 88:1 92:1 95:1 102:1 106:1 117:1 122:1
0 3:1 9:1 19:1 21:1 24:1 34:1 36:1 39:1 51:1 53:1 56:1 65:1 69:1 77:1 86:1 88:1 92:1 95:1 102:1 106:1 116:1 122:1
0 4:1 7:1 11:1 22:1 29:1 34:1 36:1 40:1 41:1 53:1 58:1 65:1 69:1 77:1 86:1 88:1 92:1 95:1 102:1 105:1 119:1 124:1
0 3:1 10:1 20:1 21:1 23:1 34:1 37:1 40:1 42:1 54:1 55:1 65:1 69:1 77:1 86:1 88:1 92:1 95:1 102:1 106:1 118:1 126:1
1 3:1 9:1 11:1 21:1 30:1 34:1 36:1 40:1 51:1 53:1 58:1 65:1 69:1 77:1 86:1 88:1 92:1 95:1 102:1 106:1 117:1 124:1

"""上面是 libsvm 的数据存储格式， 也是一种常用的格式，存储的稀疏数据。第一列是 label. a:b a 表示 index， b 表示在该 index 下的数值， 这就类似于 one-hot"""

import numpy as np
import scipy.sparse   # 稀疏矩阵的处理
import pickle
import xgboost as xgb

```
# libsvm format data 的读入方式， 直接用 xgb 的 DMatrix
dtrain = xgb.DMatrix('./xgbdata/agaricus.txt.train')
dtest = xgb.DMatrix('./xgbdata/agaricus.txt.test')
----
```

```
+*Out[54]:*+
----
[20:36:31]  6513x127  matrix  with  143286  entries  loaded
from ./xgbdata/agaricus.txt.train
[20:36:31]  1611x127  matrix  with  35442  entries  loaded
from ./xgbdata/agaricus.txt.test
----
```

```
+*In[55]:*+
[source, ipython3]
----
dtrain
----
```

```
+*Out[55]:*+
----<xgboost.core.DMatrix at 0x20114554438>----
```

```
+*In[57]:*+
[source, ipython3]
----
"""paramet setting"""
param = {
    'max_depth': 2,
    'eta': 1,
    'silent': 1,
    'objective': 'binary:logistic'
}
watch_list = [(dtest, 'eval'), (dtrain, 'train')]  # 这个是观测的时候在什么上面的结果  观测集
num_round = 5
model  =  xgb.train(params=param,  dtrain=dtrain,
num_boost_round=num_round, evals=watch_list)
----
```

+*Out[57]:*+
----
[0] eval-error:0.042831 train-error:0.046522
[1] eval-error:0.021726 train-error:0.022263
[2] eval-error:0.006207 train-error:0.007063
[3] eval-error:0.018001 train-error:0.0152
[4] eval-error:0.006207 train-error:0.007063

+*In[58]:*+
[source, ipython3]

"""预测"""
pred = model.predict(dtest)
pred   # 这里面表示的是正样本的概率是多少

+*Out[58]:*+
----array([0.08073306, 0.92217326, 0.08073306, ..., 0.98059034,
0.01182149,
       0.98059034], dtype=float32)----

+*In[59]:*+
[source, ipython3]
----
labels = dtest.get_label()
labels

+*Out[59]:*+
----array([0., 1., 0., ..., 1., 0., 1.], dtype=float32)----

+*In[60]:*+
[source, ipython3]
----
from sklearn.metrics import accuracy_score
predict_label = [round(values) for values in pred]
accuracy_score(labels, predict_label)

+*Out[60]:*+
----0.9937926753569212----

+*In[61]:*+
[source, ipython3]

"""模型存储"""
model.dump_model('xgb.model')

+*In[69]:*+
[source, ipython3]

上面的方式不知道怎么导入这个模型了，所以使用 pickle 序列化和反序列化模型
pickle.dump(model, open('./model/xgb1.pkl', 'wb'))

+*In[70]:*+
[source, ipython3]

"""导入模型"""
model1 = pickle.load(open('./model/xgb1.pkl', 'rb'))
model1.predict(dtest)

+*Out[70]:*+
----array([0.08073306, 0.92217326, 0.08073306, ..., 0.98059034, 0.01182149,
       0.98059034], dtype=float32)----

+*In[72]:*+
[source, ipython3]

"""第二种模型的存储与导入方式 - sklearn 的 joblib"""
from sklearn.externals import joblib
joblib.dump(model, './model/xgb.pkl')

```
model2 = joblib.load('./model/xgb.pkl')
model2.predict(dtest)
```

+*Out[72]:*+

```
----array([0.08073306,  0.92217326,  0.08073306,  ...,  0.98059034,
0.01182149,
       0.98059034], dtype=float32)----
```

===== 交叉验证 xgb.cv

上面是如果我指定了一组参数，我如何在这组参数下完成训练和测试，而交叉验证这个是我做模型得做到一个什么样的程度

+*In[74]:*+

```
[source, ipython3]
----
import numpy as np
import xgboost as xgb
dtrain = xgb.DMatrix('./xgbdata/agaricus.txt.train')

# 这是模型本身的参数
param       =       {'max_depth':2,       'eta':1,       'silent':1,
'objective':'binary:logistic'}
num_round = 5   # 这个是和训练相关的参数

xgb.cv(param, dtrain, num_round, nfold=5, metrics={'error'}, seed=3)
----
```

+*Out[74]:*+

```
----
[21:10:49]  6513x127  matrix  with  143286  entries  loaded
from ./xgbdata/agaricus.txt.train

[cols=",,,,",options="header",]
|=======================================================================
| |train-error-mean |train-error-std |test-error-mean |test-error-std
|0 |0.046522 |0.001200 |0.046523 |0.004801
|1 |0.022263 |0.000867 |0.022263 |0.003470
|2 |0.007063 |0.000506 |0.007062 |0.002023
|3 |0.015200 |0.001074 |0.015199 |0.004295
```

|4 |0.007063 |0.000506 |0.007062 |0.002023
|===

==== 高级点的功能

===== 调整样本权重

+*In[75]:*+
[source, ipython3]

这个函数是说在训练之前，先做一个预处理，计算一下正负样本的个数，然后加一个权重,解决样本不平衡的问题
def preproc(dtrain, dtest, param):
    labels = dtrain.get_label()
    ratio = float(np.sum(labels==0)) / np.sum(labels==1)
    param['scale_pos_ratio'] = ratio
    return (dtrain, dtest, param)

下面我们在做交叉验证， 指明 fpreproc 这个参数就可以调整样本权重
xgb.cv(param, dtrain, num_round, nfold=5, metrics={'auc'}, seed=3, fpreproc=preproc)

+*Out[75]:*+

[cols=",,,,",options="header",]
|===
| |train-auc-mean |train-auc-std |test-auc-mean |test-auc-std
|0 |0.958229 |0.000999 |0.958240 |0.004015
|1 |0.981414 |0.000701 |0.981429 |0.002824
|2 |0.997069 |0.000423 |0.997046 |0.001723
|3 |0.998758 |0.000123 |0.998769 |0.000500
|4 |0.999332 |0.000097 |0.999201 |0.000382
|===

===== 自定义目标函数（损失函数）

目标函数评价的是我和标准答案的接近程度，损失函数是我和标准答案的差异度
如果在一个比赛中，人家给了自己的评判标准，那么这时候就需要用人家的这个评判标准，这时候需要修改 xgboost 的损失函数，

但是这时候请注意一定要提供一阶和二阶导数

+*In[76]:*+
[source, ipython3]

```
# 自定义目标函数（log 似然损失），这个是逻辑回归的似然损失。 交叉验证
# 注意： 需要提供一阶和二阶导数

def logregobj(pred, dtrain):
    labels = dtrain.get_label()
    pred = 1.0 / (1+np.exp(-pred))    # sigmoid 函数
    grad = pred - labels
    hess = pred * (1-pred)
    return grad, hess     # 返回一阶导数和二阶导数
```

+*In[79]:*+
[source, ipython3]

```
def evalerror(pred, dtrain):
    labels = dtrain.get_label()
    return 'error', float(sum(labels!=(pred>0.0)))/len(labels)
```

+*In[80]:*+
[source, ipython3]

```
param = {'max_depth':2, 'eta':1, 'silent':1}

# 自定义目标函数训练
model = xgb.train(param, dtrain, num_round, watch_list, logregobj, evalerror)
```

+*Out[80]:*+

```
[0] eval-rmse:1.59228   train-rmse:1.59598  eval-error:0.042831 train-error:0.046522
[1] eval-rmse:2.4052    train-rmse:2.40976  eval-error:0.021726 train-error:0.022263
```

[2] eval-rmse:2.88253 train-rmse:2.8746 eval-error:0.006207 train-error:0.007063
[3] eval-rmse:3.62808 train-rmse:3.63615 eval-error:0.018001 train-error:0.0152
[4] eval-rmse:3.80796 train-rmse:3.83885 eval-error:0.006207 train-error:0.007063

+*In[83]:*+
[source, ipython3]

交叉验证
xgb.cv(param, dtrain, num_round, nfold=5, seed=3, obj=logregobj, feval=evalerror)

+*Out[83]:*+

[cols=",,,,,,,,",options="header",]
|===
| |train-error-mean |train-error-std |train-rmse-mean |train-rmse-std |test-error-mean |test-error-std |test-rmse-mean |test-rmse-std
|0 |0.046522 |0.001200 |1.594714 |0.001791 |0.046523 |0.004801 |1.594571 |0.017432

|1 |0.022263 |0.000867 |2.403632 |0.004994 |0.022263 |0.003470 |2.403480 |0.030402

|2 |0.007063 |0.000506 |2.856645 |0.011636 |0.007062 |0.002023 |2.855064 |0.033166

|3 |0.015200 |0.001074 |3.619410 |0.016135 |0.015199 |0.004295 |3.617991 |0.029253

|4 |0.005988 |0.002618 |3.804485 |0.030290 |0.005527 |0.001488 |3.810884 |0.048982
|===

===== 用前 n 棵树做预测 ntree_limit

太多的树可能发生过拟合，这时候我们可以指定前 n 棵树做预测

+*In[84]:*+
[source, ipython3]

前 1 棵
pred1 = model.predict(dtest, ntree_limit=1)
evalerror(pred2, dtest)

+*Out[84]:*+
----('error', 0.04283054003724395)----

+*In[88]:*+
[source, ipython3]

前 2 棵
pred2 = model.predict(dtest, ntree_limit=2)
evalerror(pred2, dtest)

+*Out[88]:*+
----('error', 0.021725636250775917)----

+*In[89]:*+
[source, ipython3]

前 3 棵
pred3 = model.predict(dtest, ntree_limit=3)
evalerror(pred3, dtest)

+*Out[89]:*+
----('error', 0.006207324643078833)----

===== 画出特征重要度

+*In[90]:*+
[source, ipython3]
----
from xgboost import plot_importance

plot_importance(model, max_num_features=10)

+*Out[90]:*+
----<matplotlib.axes._subplots.AxesSubplot at 0x2011407ef98>
![png](output_125_1.png)

==== 如何和 sklearn/Pandas 结合

+*In[99]:*+
[source, ipython3]
----
import pickle
import xgboost as xgb
import numpy as np
from sklearn.model_selection import KFold, train_test_split, GridSearchCV
from sklearn.metrics import confusion_matrix, mean_squared_error
from sklearn.datasets import load_iris, load_digits, load_boston

import warnings
warnings.filterwarnings('ignore')

+*In[92]:*+
[source, ipython3]

#用 Xgboost 建模，用 sklearn 做评估
#二分类问题，用混淆矩阵

digits = load_digits()
y = digits['target']
X = digits['data']

+*In[93]:*+
[source, ipython3]

X.shape

+*Out[93]:*+
----(1797, 64)----

+*In[94]:*+
[source, ipython3]

K 折切分器
kf = KFold(n_splits=2, shuffle=True, random_state=1234)
for train_index, test_index in kf.split(X):
    xgboost_model = xgb.XGBClassifier().fit(X[train_index],
y[train_index])
    pred = xgboost_model.predict(X[test_index])
    groud_truth = y[test_index]
    print(confusion_matrix(groud_truth, pred))

+*Out[94]:*+

[[78 0 0 0 0 0 0 0 1 0]
 [0 92 1 0 0 0 0 0 0 0]
 [0 2 82 0 0 0 2 0 0 0]
 [0 1 1 88 0 0 0 1 0 3]
 [2 0 0 0 99 0 2 3 1 0]
 [0 0 0 1 0 95 2 0 0 4]
 [0 2 0 0 0 0 84 0 2 0]
 [0 0 0 0 0 0 0 86 0 2]
 [0 6 0 2 0 0 0 0 73 1]
 [0 1 0 0 1 0 0 5 2 71]]
[[98 0 0 0 0 0 0 1 0 0]
 [0 84 2 1 0 0 1 0 0 1]
 [1 0 88 0 0 0 0 1 1 0]
 [0 0 1 86 0 1 0 0 0 1]
 [0 0 0 0 74 0 0 0 0 0]

```
 [ 1  0  0  0  1 73  0  0  1  4]
 [ 0  0  0  0  1  1 91  0  0  0]
 [ 0  0  0  1  0  0  0 89  1  0]
 [ 1  1  0  0  1  1  0  0 87  1]
 [ 0  2  0  1  0  1  0  0  2 94]]
---
```

```
+*In[95]:*+
[source, ipython3]
---
#多分类
iris = load_iris()
y_iris = iris['target']
X_iris = iris['data']
kf = KFold(n_splits=2, shuffle=True, random_state=1234)
for train_index, test_index in kf.split(X_iris):
    xgboost_model    =    xgb.XGBClassifier().fit(X_iris[train_index],
y_iris[train_index])
    #预测结果
    pred = xgboost_model.predict(X_iris[test_index])
    #标准答案
    ground_truth = y_iris[test_index]
    print(confusion_matrix(ground_truth, pred))
---
```

```
+*Out[95]:*+
---
[[25  0  0]
 [ 0 24  1]
 [ 0  0 25]]
[[25  0  0]
 [ 0 23  2]
 [ 0  2 23]]
---
```

```
+*In[96]:*+
[source, ipython3]
---
#回归问题
boston = load_boston()
y_boston = boston['target']
```

```
X_boston = boston['data']
kf = KFold(n_splits=2, shuffle=True, random_state=1234)
for train_index, test_index in kf.split(X_boston):
    xgboost_model    =    xgb.XGBRegressor().fit(X_boston[train_index],
y_boston[train_index])
    #预测结果
    pred = xgboost_model.predict(X_boston[test_index])
    #标准答案
    ground_truth = y_boston[test_index]
    print(mean_squared_error(ground_truth, pred))
----
```

```
+*Out[96]:*+
----
[21:47:02]              WARNING:              d:\build\xgboost\xgboost-
0.90.git\src\objective\regression_obj.cu:152:    reg:linear    is    now
deprecated in favor of reg:squarederror.
10.10385764511765
[21:47:02]              WARNING:              d:\build\xgboost\xgboost-
0.90.git\src\objective\regression_obj.cu:152:    reg:linear    is    now
deprecated in favor of reg:squarederror.
15.024626548564884
----
```

==== 优化超参数

```
+*In[100]:*+
[source, ipython3]
----
y_boston = boston['target']
X_boston = boston['data']
xgb_model = xgb.XGBRegressor()
#参数字典
param_dict = {'max_depth':[2,4,6], 'n_estimators':[50, 100, 200]}

rgs = GridSearchCV(xgb_model, param_dict)

rgs.fit(X_boston, y_boston)
----
```

```
+*Out[100]:*+
```

```
[21:49:43]              WARNING:              d:\build\xgboost\xgboost-
0.90.git\src\objective\regression_obj.cu:152:    reg:linear    is    now
deprecated in favor of reg:squarederror.
[21:49:43]              WARNING:              d:\build\xgboost\xgboost-
0.90.git\src\objective\regression_obj.cu:152:    reg:linear    is    now
deprecated in favor of reg:squarederror.
[21:49:43]              WARNING:              d:\build\xgboost\xgboost-
0.90.git\src\objective\regression_obj.cu:152:    reg:linear    is    now
deprecated in favor of reg:squarederror.
[21:49:43]              WARNING:              d:\build\xgboost\xgboost-
0.90.git\src\objective\regression_obj.cu:152:    reg:linear    is    now
deprecated in favor of reg:squarederror.
[21:49:43]              WARNING:              d:\build\xgboost\xgboost-
0.90.git\src\objective\regression_obj.cu:152:    reg:linear    is    now
deprecated in favor of reg:squarederror.
[21:49:43]              WARNING:              d:\build\xgboost\xgboost-
0.90.git\src\objective\regression_obj.cu:152:    reg:linear    is    now
deprecated in favor of reg:squarederror.
[21:49:43]              WARNING:              d:\build\xgboost\xgboost-
0.90.git\src\objective\regression_obj.cu:152:    reg:linear    is    now
deprecated in favor of reg:squarederror.
[21:49:43]              WARNING:              d:\build\xgboost\xgboost-
0.90.git\src\objective\regression_obj.cu:152:    reg:linear    is    now
deprecated in favor of reg:squarederror.
[21:49:43]              WARNING:              d:\build\xgboost\xgboost-
0.90.git\src\objective\regression_obj.cu:152:    reg:linear    is    now
deprecated in favor of reg:squarederror.
[21:49:43]              WARNING:              d:\build\xgboost\xgboost-
0.90.git\src\objective\regression_obj.cu:152:    reg:linear    is    now
deprecated in favor of reg:squarederror.
[21:49:43]              WARNING:              d:\build\xgboost\xgboost-
0.90.git\src\objective\regression_obj.cu:152:    reg:linear    is    now
deprecated in favor of reg:squarederror.
[21:49:43]              WARNING:              d:\build\xgboost\xgboost-
0.90.git\src\objective\regression_obj.cu:152:    reg:linear    is    now
deprecated in favor of reg:squarederror.
[21:49:44]              WARNING:              d:\build\xgboost\xgboost-
```

0.90.git\src\objective\regression_obj.cu:152: reg:linear is now deprecated in favor of reg:squarederror.
[21:49:44] WARNING: d:\build\xgboost\xgboost-0.90.git\src\objective\regression_obj.cu:152: reg:linear is now deprecated in favor of reg:squarederror.
[21:49:44] WARNING: d:\build\xgboost\xgboost-0.90.git\src\objective\regression_obj.cu:152: reg:linear is now deprecated in favor of reg:squarederror.
[21:49:44] WARNING: d:\build\xgboost\xgboost-0.90.git\src\objective\regression_obj.cu:152: reg:linear is now deprecated in favor of reg:squarederror.
[21:49:44] WARNING: d:\build\xgboost\xgboost-0.90.git\src\objective\regression_obj.cu:152: reg:linear is now deprecated in favor of reg:squarederror.
[21:49:44] WARNING: d:\build\xgboost\xgboost-0.90.git\src\objective\regression_obj.cu:152: reg:linear is now deprecated in favor of reg:squarederror.
[21:49:44] WARNING: d:\build\xgboost\xgboost-0.90.git\src\objective\regression_obj.cu:152: reg:linear is now deprecated in favor of reg:squarederror.
[21:49:44] WARNING: d:\build\xgboost\xgboost-0.90.git\src\objective\regression_obj.cu:152: reg:linear is now deprecated in favor of reg:squarederror.
[21:49:44] WARNING: d:\build\xgboost\xgboost-0.90.git\src\objective\regression_obj.cu:152: reg:linear is now deprecated in favor of reg:squarederror.
[21:49:44] WARNING: d:\build\xgboost\xgboost-0.90.git\src\objective\regression_obj.cu:152: reg:linear is now deprecated in favor of reg:squarederror.
[21:49:44] WARNING: d:\build\xgboost\xgboost-0.90.git\src\objective\regression_obj.cu:152: reg:linear is now deprecated in favor of reg:squarederror.
[21:49:44] WARNING: d:\build\xgboost\xgboost-0.90.git\src\objective\regression_obj.cu:152: reg:linear is now deprecated in favor of reg:squarederror.
[21:49:44] WARNING: d:\build\xgboost\xgboost-0.90.git\src\objective\regression_obj.cu:152: reg:linear is now deprecated in favor of reg:squarederror.
[21:49:44] WARNING: d:\build\xgboost\xgboost-0.90.git\src\objective\regression_obj.cu:152: reg:linear is now deprecated in favor of reg:squarederror.
[21:49:44] WARNING: d:\build\xgboost\xgboost-0.90.git\src\objective\regression_obj.cu:152: reg:linear is now deprecated in favor of reg:squarederror.
GridSearchCV(cv='warn', error_score='raise-deprecating',
             estimator=XGBRegressor(base_score=0.5, booster='gbtree',
                                    colsample_bylevel=1,

```
colsample_bynode=1,
                                colsample_bytree=1, gamma=0,
                                importance_type='gain',
learning_rate=0.1,
                                max_delta_step=0, max_depth=3,
                                min_child_weight=1, missing=None,
                                n_estimators=100,         n_jobs=1,
nthread=None,
                                objective='reg:linear',
random_state=0,
                                reg_alpha=0, reg_lambda=1,
                                scale_pos_weight=1,      seed=None,
silent=None,
                                subsample=1, verbosity=1),
             iid='warn', n_jobs=None,
             param_grid={'max_depth': [2, 4, 6],
                         'n_estimators': [50, 100, 200]},
             pre_dispatch='2*n_jobs',                         refit=True,
return_train_score=False,
             scoring=None, verbose=0)----


+*In[98]:*+
[source, ipython3]
----
print(rgs.best_score_)
print(rgs.best_params_)
----


+*Out[98]:*+
----
0.6001029721598573
{'max_depth': 4, 'n_estimators': 100}
----
```

=== 一些 tricks

这里主要是记录几个比赛的相关 tricks：

_____

1. 拿到数据之后，一定要从质疑的态度去审视数据，看看有没有个别的数值不合法（这里可以借助 Excel 的套用表格样式，或者自己写一个 python 函数遍历进行检查审视）
2. 数据预处理的时候，标准化归一化这种，最好是把测试集和训练集放在一块进行处理，这样可以更方便，前提得先统一维度，即把训练集的标签单独存放，然后再两者进行合并处理，合并的时候要记录训练集样本个数。处理完之后，再分开。
3. 进行数据处理的时候，最好是不要在原数据上进行处理，写成函数，然后进行好备份操作。
4. 做特征工程的时候，如果是从不同的角度构建特征，我们最好是每个角度写成函数创建一个空的 DataFrame 单独构建，

最后利用统一 index 进行合并

5. 关于数据的时间字段，处理的时候最好是给重命名一下，这样方便后面特殊处理的时候好找，或者还可以利用 to_datetime 转成标准的时间，这样这个字段就可以做很多事情了，也可以设置成索引等好多处理方式。
6. 时间段字段，可以从上面重新构建新特征，比如起始时间，终止时间，持续时间等。如果计算持续时间的时候，普通的表示方式计算机并不认识，这时候我们可以考虑转成分钟或者小时的方式。
7. 做特征工程的时候，我们需要发挥想象力，尽可能多的创造特征，不用先考虑哪些特征可能好，可能不好，先从弥补广度出发，做出来尽可能多的，不好的特征我们可以再后期删除嘛。

所以我们不仅只做一些简单的加减组合，我们还要考虑提取一些统计的特征。当然，这是连续的数值，如果是离散的数据，后面也会有相应的方式。

8. 筛选不常规的数据，我们观察数据的时候，也可以进行数据可视化，筛除掉一些离群的数据样本先。

_____

_____

_____

_____

_____